

RWTH Aachen
Faculty of Mathematics and Natural Sciences
Institute for Quantum Information
Bachelor's Thesis

Efficient Coordinate Transformations for Permutation-Invariant Quantum Systems

Lennart Sauer

Supervisor:	Professor Dr. Mario Berta
Second Supervisor:	Professor Dr. Fabian Hassler
Advisors:	Dr. Robert Salzmann & Julius Zeiss

A thesis submitted in partial fulfillment of the requirements
for the degree of B.Sc. in Physics

September 2026

Abstract

Permutation symmetry reduces the degrees of freedom of an n -qubit operator from 4^n to only $O(n^3)$ parameters. Exploiting this reduction in practice requires efficient conversion between different, symmetry-adapted representations. We develop classical algorithms for converting between three such bases: the symmetrized Pauli and matrix-unit bases, as well as the block-diagonal Schur basis. Our construction builds on prior work, composing a Krawtchouk transform with combinatorial results from coding theory. We obtain algorithms with a complexity within a linear factor of the output size of the problem and provide a tested Python implementation. In particular, our Pauli–Schur transformation reduces the arithmetic cost of the prior tensor-network construction of Anschuetz et al. from $O(n^7)$ to $O(n^4)$, including preprocessing. We use these conversions to compute eigenenergies and the time evolution of permutation-invariant observables.

Contents

Abstract	i
1 Introduction	1
1.1 Motivation and problems	1
1.2 Overview of the main contributions	2
1.3 Structure of this thesis	3
2 Preliminaries	4
2.1 Multi-qubit matrix spaces and sparsity relations	4
2.1.1 The space $\text{End}(\mathcal{H}_n)$ of linear operators on n qubits	4
2.1.2 Sparsity and locality notions	6
2.2 Permutation symmetry and invariant matrices	7
2.2.1 The symmetric group and its action on $\mathcal{H}_n$	8
2.2.2 Permutation-invariant operators and orbit bases	8
2.2.3 Schur blocks and Schur basis	11
3 Existing approaches and related work	13
3.1 General and sparse Pauli decomposition	13
3.1.1 Pauli decomposition of arbitrary matrices using the fast Walsh-Hadamard transform	13
3.1.2 Pauli decomposition of sparse matrices	14
3.2 Methods for permutation-invariant operators	16
3.2.1 Schur block construction via tensor networks	16
3.2.2 Direct block construction for fixed-locality generators	17
3.2.3 Recent results by Bergh and Parentin and prior work by Muñoz et al.	18
3.3 Comparison of existing approaches and our results	19
4 Matrix-unit orbits and Pauli orbits	21
4.1 Deriving the efficient transformation	21
4.1.1 The one-qubit change of basis	21
4.1.2 Restriction to symmetric tensors	22
4.1.3 The separated Krawtchouk transform	23
4.1.4 The inverse transform	24
4.2 Complexity analysis	25
4.2.1 Naïve implementation	25
4.2.2 Fast matrix multiplication improvement	26
5 Schur blocks and matrix-unit orbits	27
5.1 Deriving the efficient transformation	27
5.1.1 Choice of coordinates	27

5.1.2	The explicit transformation formula	28
5.2	Complexity analysis	29
5.2.1	Naive implementation	29
5.2.2	Fast matrix multiplication improvements	30
5.3	The composed Schur-Pauli transformation	31
6	Applications	33
6.1	Spectral computations	33
6.1.1	Eigenvalues and eigenstates	33
6.1.2	Computing only selected eigenvalues	34
6.2	Invariant dynamics	34
6.2.1	Classically supplied state data	35
6.2.2	Access through quantum copies	36
6.3	Numerical applications	37
6.3.1	Random spectral computations	37
6.3.2	Random invariant dynamics	38
6.3.3	The fully connected Ising model	38
7	Conclusion and outlook	41
7.1	Conclusion	41
7.2	Outlook	41
7.2.1	Fast polynomial transforms	41
7.2.2	Generalizing to qudits	42
7.2.3	Generalizing to other groups	42
7.2.4	Sparse Fourier reconstruction	42
7.2.5	Quantum channel fidelity and semidefinite programming	43
	Bibliography	44
A	Group and representation theory	49
A.1	Group orbits	49
A.2	Basic representation theory concepts and proofs	49
A.3	The permutation representation	50
A.4	Proving the invariant block form	51
A.5	Proof of Schur basis properties	52
A.6	Spectral consequences of the Schur decomposition	53
A.7	Other symmetry groups	53
A.7.1	Separate permutations of two sets	53
A.7.2	Cyclic translations	54
B	Sparsity and transformation details	55
B.1	The Pauli basis	55
B.2	Pauli row-sparsity and computational sparsity	55
B.3	A factorization of the Schur conversion coefficients	56
C	Algorithms and numerical comparisons	57
C.1	Pseudocode for the coordinate transformations	57
C.1.1	Matrix-unit and Pauli orbit coefficients	57
C.1.2	Matrix-unit orbit coefficients and Schur blocks	58
C.1.3	Remarks on the Python implementation	59
C.2	Implementation and numerical comparisons	59

C.2.1	Implementation	59
C.2.2	General coordinate conversion	60
C.2.3	Matrix-unit orbit to Schur conversion	60
C.2.4	Fixed-locality comparison	61
C.2.5	Accuracy of the coordinate conversions	61
C.3	Application benchmark details	62
C.3.1	Application accuracy	62
C.4	Additional benchmark figures	63
AI Disclosure		65
Acknowledgments		66

Chapter 1

Introduction

1.1 Motivation and problems

An operator A on n qubits can be represented by its matrix entries in the computational basis or by its coefficients in the Pauli decomposition

$$A = \sum_{P \in \mathcal{P}_n} \alpha_P P, \quad \mathcal{P}_n = \{P_1 \otimes \cdots \otimes P_n : P_j \in \{I, X, Y, Z\}\}. \quad (1.1)$$

As a prototypical linear combination of unitaries, this decomposition is used in quantum algorithms in Hamiltonian simulation, quantum chemistry and spectral estimation [Dal+25; Geo+25b; WBC22; WMB24]. Obtaining these Pauli coefficients from another representation is classical preprocessing whose cost must enter into the analysis of end-to-end quantum advantage [Dal+25; WMB24].

For a general operator, both representations contain 4^n independent coordinates, so computing all Pauli coefficients necessarily scales exponentially. The fast Walsh-Hadamard approach of Georges et al. achieves an arithmetic complexity of $O(n4^n)$ [Geo+25a]. Polynomial-time conversion requires additional assumptions and properly adapted representations, for example through sparsity, locality or symmetry. We focus on the case of permutation symmetry and briefly review some prior approaches which utilize the other two properties.

Let the *symmetric group* S_n act by permuting the qubits, represented as unitaries U_π for $\pi \in S_n$. An operator is *permutation invariant* if $U_\pi A U_\pi^\dagger = A$ for every $\pi \in S_n$, as is the case for example for Hamiltonians of fully-connected spin models with uniform coupling [LMG65]. The S_n action permutes the tensor product factors in a given Pauli string, so α_P depends only on the numbers of I , X , Y and Z factors. These counts must sum to n , leaving only $\binom{n+3}{3} = O(n^3)$ independent coefficients. Grouping the Pauli strings by these common coefficients gives the *Pauli orbit basis*, whose elements are for example also used in permutationally invariant tomography [T6t+10].

The same construction applied to the matrix units $|u\rangle\langle v|$ gives a *matrix-unit orbit basis* for the computational-basis matrix entries. With classical query access to these matrix entries, querying one representative per orbit supplies the compressed input efficiently.

A third description follows from *Schur–Weyl duality*. In the Schur basis, every invariant operator has a block diagonal form

$$A \cong \bigoplus_{k=0}^{\lfloor n/2 \rfloor} H_k \otimes I_{\mu_k}, \quad H_k \in \mathbb{C}^{d_k \times d_k}, \quad d_k = n - 2k + 1, \quad (1.2)$$

where μ_k is the multiplicity of the block H_k . Storing one copy of each block again requires only $O(n^3)$ coordinates. This representation reduces spectral computations to

matrices of dimension at most $n + 1$ and allows the evaluation of invariant dynamics when appropriate initial state data is available [Ans+23; CLC26]. Positivity can likewise be imposed separately on each block, a property used in permutation-invariant optimization for quantum communication [BP26c].

The above descriptions may serve different tasks. A Hamiltonian specified by Pauli interactions may be diagonalized through its Schur block form efficiently. Conversely, an evolved observable may require Pauli coordinates for quantum implementation. We study efficient conversion between the three compressed descriptions, starting from any one of them.

1.2 Overview of the main contributions

We construct classical algorithms for converting between all three representations in all directions. Each conversion uses $O(n^4)$ arithmetic operations, including preprocessing, under a *unit-cost arithmetic model*. If two $d \times d$ matrices can be multiplied in $O(d^\tau)$ operations¹, the bound becomes $O(n^{1+\tau})$. The complete output already contains $\Theta(n^3)$ coordinates, so our bound is within a linear factor of this lower bound. Figure 1.1 summarizes the three conversions.

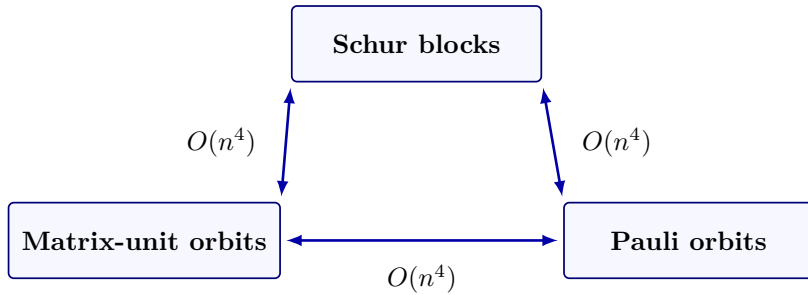


Figure 1.1: Bidirectional conversions between three compressed descriptions of a permutation-invariant operator. Each conversion requires $O(n^4)$ arithmetic operations, including preprocessing, or $O(n^{1+\tau})$ using fast matrix multiplication.

The matrix-unit/Pauli conversion restricts the general one-qubit change of basis to symmetric tensor powers. For the Hadamard matrix, which is essentially the one-qubit change of basis matrix, this gives Krawtchouk transforms [FK07], which are then evaluated via matrix multiplications. For the conversion between matrix-unit orbit coefficients and Schur blocks, we use the block-diagonalization formulas of Gijswijt and Vallentin [Gij09; Val09].

Composing the two transformations gives the Pauli–Schur conversion. For Pauli orbit input and Schur block output, our bound improves on the tensor-network construction of Anschuetz et al., which requires $O(n^7)$ preprocessing and $O(n^6)$ operations for each following conversion [Ans+23]. We apply our results to spectral computations and invariant dynamics and compare our implementation numerically with existing approaches, including an optimized version of Anschuetz et al.’s combinatorial construction. The accompanying Python implementation is available at [Sau26].

¹For example, Strassen’s algorithm gives $\tau = \log_2 7$ [Str69], while recent results by OpenAI [Ope26b] give an upper bound of $\tau = 9/4$.

1.3 Structure of this thesis

After the preliminaries, [Chapter 3](#) compares prior approaches. In [Chapters 4](#) and [5](#) we then derive the main coordinate conversions, while [Chapter 6](#) develops their applications and presents numerical examples. [Chapter 7](#) summarizes the results and discusses some improvements and generalizations.

Chapter 2

Preliminaries

In this chapter, we recap the necessary mathematical background and fix notation and conventions. We assume familiarity with basic notions of linear algebra, group theory and representation theory, such as vector spaces, group representations, linear maps and tensor products. Additionally, when considering applications, we assume some familiarity with the usual axioms of quantum mechanics. A standard, exhaustive reference for the representation theory of finite groups is [FH13], while for the necessary background on quantum information, we refer to [NC00]. Two resources we found particularly useful for the representation theory of the symmetric group are [Sag91] and [Gri25]. A more detailed discussion of the representation-theoretic background, including proofs of some of the relevant results, is also given in [Appendix A](#).

Unless explicitly stated otherwise, we consider all groups to be finite and all vector spaces to be finite-dimensional over the complex numbers. The words matrix and operator are thus used synonymously.

2.1 Multi-qubit matrix spaces and sparsity relations

2.1.1 The space $\text{End}(\mathcal{H}_n)$ of linear operators on n qubits

The tensor product postulate of quantum mechanics states that the Hilbert space representing a composite quantum system is given by the tensor product of the Hilbert spaces of its subsystems¹. In particular, the Hilbert space of an n -qubit system is given by the n -fold tensor product of the Hilbert space of a single qubit, i.e.

$$\mathcal{H}_n \cong (\mathbb{C}^2)^{\otimes n} \cong \mathbb{C}^{2^n}.$$

Remark 2.1. In the following, we mainly restrict our discussion to the case of qubits, but many of our results may be extendable to qudits, for which the Hilbert space is given by $\mathcal{H}_n \cong (\mathbb{C}^d)^{\otimes n} \cong \mathbb{C}^{d^n}$ (see [Section 7.2](#)). Additionally, as we are working in the finite-dimensional setting, the abstract tensor product may be replaced by the Kronecker product of matrices [Van00], which is the usual matrix representation of the tensor product of linear maps.

The *computational basis* of $\mathcal{H}_n$ is given by the set of 2^n vectors

$$\{|i_1\rangle \otimes \cdots \otimes |i_n\rangle : i_1, \dots, i_n \in \{0, 1\}\},$$

¹Modern work has shown that this fact is not actually a postulate, but may be derived from the other postulates of quantum mechanics [CMA21].

where $|0\rangle = (1, 0)^T$ and $|1\rangle = (0, 1)^T$ are the standard basis vectors of $\mathbb{C}^2$. For a binary string $u = (u_1, \dots, u_n) \in \{0, 1\}^n$, we write $|u\rangle = |u_1 \dots u_n\rangle := |u_1\rangle \otimes \dots \otimes |u_n\rangle$. Thus binary strings serve as labels for the computational basis vectors. We denote the space of linear operators (endomorphisms) on this Hilbert space by $\text{End}(\mathcal{H}_n) \cong \mathbb{C}^{2^n \times 2^n}$. Using the usual matrix multiplication, we may turn this space into an inner product space by defining the *Hilbert-Schmidt inner product*

$$\langle A, B \rangle_{HS} := \text{Tr}(A^\dagger B),$$

where $A^\dagger$ denotes the conjugate transpose of A . The necessary properties of this inner product are straightforward to verify. The induced norm is called the *Hilbert-Schmidt norm* and is given by

$$\|A\|_{HS} := \sqrt{\langle A, A \rangle_{HS}} = \sqrt{\text{Tr}(A^\dagger A)}.$$

With the usual matrix multiplication, the conjugate transpose as the involution and the operator norm $\|A\|_\infty := \sup_{\|\psi\|=1} \|A\psi\|$, $\text{End}(\mathcal{H}_n)$ is a C^* -algebra over $\mathbb{C}$ (we refer to [Arv76] for a precise definition and introduction). We will sometimes consider the real subspace of Hermitian operators (that is, those satisfying $A^\dagger = A$), which is of particular interest in quantum mechanics, as its elements correspond exactly to the observables of our system.

Three important identities of the tensor product, in addition to its bilinearity, are the following:

$$(A \otimes B)(C \otimes D) = (AC) \otimes (BD), \quad (2.1)$$

$$(A \otimes B)^\dagger = A^\dagger \otimes B^\dagger, \quad (2.2)$$

$$\text{Tr}(A \otimes B) = \text{Tr}(A) \text{Tr}(B). \quad (2.3)$$

The dimension of $\text{End}(\mathcal{H}_n)$ is equal to $2^n \cdot 2^n = 4^n$, and thus grows exponentially in the number of qubits. A natural basis of this space is given by the *matrix units* $E_{u,v}$, where $u, v \in \{0, 1\}^n$ label computational basis vectors. We index rows and columns by the same binary strings and define

$$(E_{u,v})_{i,j} := \delta_{i,u} \delta_{j,v}, \quad (2.4)$$

where $i, j \in \{0, 1\}^n$ and $\delta_{i,j}$ is the Kronecker delta. Thus $E_{u,v}$ has a single 1 in row u and column v , and zeros elsewhere. In Dirac notation, this is the outer product

$$E_{u,v} = |u\rangle\langle v|.$$

We may sometimes refer to this basis as the *computational basis* of the space $\text{End}(\mathcal{H}_n)$, as it is the natural basis induced by the computational basis of $\mathcal{H}_n$. Another relevant basis is given by the set of n -fold tensor products of the *Pauli matrices*, which are defined as

$$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}. \quad (2.5)$$

Simple calculations prove the following two propositions, which we will use throughout this thesis. Alternatively, they may be found in standard references on quantum information, such as [NC00].

Proposition 2.2. *The Pauli matrices are unitary and Hermitian, which implies*

$$P^2 = I, \quad P^\dagger = P, \quad \forall P \in \{I, X, Y, Z\}. \quad (2.6)$$

Additionally, they are traceless, except for the identity matrix, which has trace 2. They satisfy:

$$XY = iZ, \quad YZ = iX, \quad ZX = iY. \quad (2.7)$$

The set of n -fold tensor products of Pauli matrices is denoted by

$$\mathcal{P}_n := \left\{ \bigotimes_{i=1}^n P_i : P_i \in \{I, X, Y, Z\} \right\}. \quad (2.8)$$

We may refer to elements of this set as *Pauli strings*, and to the set itself as the *Pauli basis*. The following lemma shows that this set indeed forms a basis of the space of linear operators on $\mathcal{H}_n$.

Lemma 2.3. *The set $\mathcal{P}_n$ forms a basis of the complex vector space $\text{End}(\mathcal{H}_n)$, as well as of the real vector space of Hermitian operators on $\mathcal{H}_n$.*

The proof is given in [Section B.1](#).

2.1.2 Sparsity and locality notions

This subsection introduces some basic relations between the matrix-unit and Pauli bases. Most of this information is well known, but we include it here for completeness and to fix notation. We start by fixing the notation for certain binary operations, which we will use to define a binary representation of Pauli strings.

Definition 2.4. The *bitwise AND* of two binary strings $x, y \in \{0, 1\}^n$ is defined as the binary string $x \wedge y \in \{0, 1\}^n$ given by

$$(x \wedge y)_i := x_i y_i, \quad \forall i \in \{1, \dots, n\}. \quad (2.9)$$

The *bitwise OR* of two binary strings $x, y \in \{0, 1\}^n$ is defined as the binary string $x \vee y \in \{0, 1\}^n$ given by

$$(x \vee y)_i := x_i + y_i - x_i y_i, \quad \forall i \in \{1, \dots, n\}. \quad (2.10)$$

The *bitwise XOR* of two binary strings $x, y \in \{0, 1\}^n$ is defined as the binary string $x \oplus y \in \{0, 1\}^n$ given by

$$(x \oplus y)_i := x_i + y_i - 2x_i y_i, \quad \forall i \in \{1, \dots, n\}. \quad (2.11)$$

The *Hamming weight* of a binary string $x \in \{0, 1\}^n$ is defined as the number of 1s in x , i.e.

$$|x| := \sum_{i=1}^n x_i. \quad (2.12)$$

Remark 2.5. Clearly, binary strings of length n have a natural representation as elements of the vector space $\mathbb{F}_2^n$ over the finite field $\mathbb{F}_2$. In this representation, the bitwise AND and XOR operations correspond to the usual multiplication and addition in $\mathbb{F}_2$, respectively. The Hamming weight then corresponds to the number of non-zero entries in a vector.

Proposition 2.6 (Binary representation of Pauli matrices). *Pauli strings may be labeled using the following representation: Let $x, z \in \{0, 1\}^n$ be two binary strings of length n , with $x = (x_1, \dots, x_n)$ and $z = (z_1, \dots, z_n)$. Then we may define a Pauli string $P(x, z) \in \mathcal{P}_n$ by*

$$P(x, z) := (-i)^{|x \wedge z|} \bigotimes_{i=1}^n Z^{z_i} X^{x_i}. \quad (2.13)$$

Proof. See [Geo+25a, Theorem 1] for a proof of this fact. $\square$

Definition 2.7 (Pauli weight). The *Pauli weight* or *locality* of a Pauli string $P \in \mathcal{P}_n$ is defined as the number of non-identity factors in the tensor product representation of P . Equivalently, if $P = P(x, z)$ for some $x, z \in \{0, 1\}^n$, then the Pauli weight of P is given by

$$\text{wt}(P) := |x \vee z| = |x| + |z| - |x \wedge z|. \quad (2.14)$$

Remark 2.8. The equivalence of the two definitions of Pauli weight follows from the fact that the identity matrix is the only Pauli matrix with both $x_i = 0$ and $z_i = 0$.

Theorem 2.9 (Pauli row-sparsity implies computational sparsity). *Let $A \in \text{End}(\mathcal{H}_n)$ with Pauli decomposition*

$$A = \sum_{x, z \in \{0, 1\}^n} \alpha_{x, z} P(x, z), \quad \alpha_{x, z} \in \mathbb{C}. \quad (2.15)$$

Then the maximum number of non-zero entries in any row or column of A (in the computational basis) is bounded above by the number of non-zero rows in the corresponding matrix of Pauli coefficients $(\alpha_{x, z})_{x, z \in \{0, 1\}^n}$.

Proof. A proof is given in Section B.2. $\square$

Interestingly, the upper bound may not be replaced by the number of non-zero columns in the matrix of Pauli coefficients, as the following example shows.

Example 2.10. Let $n = 2$ and label the binary strings (x_1, x_2) by the integers $x_1 + 2x_2$, so that 0, 1, 2, 3 denote 00, 10, 01, 11, respectively. Consider the following matrix of Pauli coefficients:

$$(\alpha_{x, z})_{x, z \in \{0, 1, 2, 3\}} = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}. \quad (2.16)$$

The corresponding operator $A \in \text{End}(\mathcal{H}_2)$ is

$$A = P(0, 1) + P(3, 1) = Z \otimes I + Y \otimes X = \begin{pmatrix} 1 & 0 & 0 & -i \\ 0 & 1 & -i & 0 \\ 0 & i & -1 & 0 \\ i & 0 & 0 & -1 \end{pmatrix}. \quad (2.17)$$

The matrix A has two non-zero entries in each row and column, whereas its matrix of Pauli coefficients has two non-zero rows but only one non-zero column.

We note that while this theorem has, to the best of our knowledge, not been stated explicitly in the literature before, it is a direct consequence of prior results, such as those in [Geo+25a] and [SBG26a]. Prior authors likely did not deem it necessary to state it in the above manner.

2.2 Permutation symmetry and invariant matrices

While sparsity and locality constitute interesting avenues for efficient Pauli decomposition, our main results achieve efficiency through the use of symmetries of the quantum system. The group of permutations of the subsystems (i.e. qubits) of a system is a natural starting point for such considerations and is thus introduced in the following section.

2.2.1 The symmetric group and its action on $\mathcal{H}_n$

Definition 2.11 (Symmetric group). The *symmetric group* S_n on n elements is the group consisting of all permutations of the set $\{1, \dots, n\}$, with group operation given by composition of permutations. The order of S_n , that is, the number of elements in the group, is equal to $n!$.

Definition 2.12 (Group action of S_n on $\mathcal{H}_n$). The symmetric group S_n acts on the Hilbert space $\mathcal{H}_n$ by permuting the tensor factors. More precisely, for a permutation $\pi \in S_n$ and a basis vector $|i_1\rangle \otimes \dots \otimes |i_n\rangle \in \mathcal{H}_n$, we define

$$\pi(|i_1\rangle \otimes \dots \otimes |i_n\rangle) := |i_{\pi^{-1}(1)}\rangle \otimes \dots \otimes |i_{\pi^{-1}(n)}\rangle. \quad (2.18)$$

This action extends linearly to all of $\mathcal{H}_n$ (and therefore constitutes a linear action, that is, a representation, of S_n on $\mathcal{H}_n$).

Corollary 2.13. *It follows easily from the definition that this action constitutes a group homomorphism from S_n to the group of unitary operators on $\mathcal{H}_n$, and therefore defines a unitary representation of S_n on $\mathcal{H}_n$.*

The proof is given in [Section A.3](#). From now on, we denote the above representation of a permutation $\pi \in S_n$ on $\mathcal{H}_n$ by U_π .

A representation of a group G on a vector space V naturally induces a representation of G on the space of linear operators $\text{End}(V)$, which is given by conjugation [\[FH13\]](#). In our case, this means that the representation of S_n on $\mathcal{H}_n$ induces a representation of S_n on $\text{End}(\mathcal{H}_n)$, which is given by

$$\pi(A) := U_\pi A U_\pi^\dagger, \quad \forall A \in \text{End}(\mathcal{H}_n), \pi \in S_n. \quad (2.19)$$

Remark 2.14. The U_π matrices are real, so $U_\pi^\dagger = U_\pi^T$. We stick to $U_\pi^\dagger$ for notational consistency.

2.2.2 Permutation-invariant operators and orbit bases

Definition 2.15. For any group G with a representation on a vector space V^2 , we may define the subspace of G -invariant vectors in V as

$$V^G := \{v \in V : g(v) = v, \forall g \in G\}. \quad (2.20)$$

By linearity of the representation, this is indeed a subspace of V .

In our case, we may define the subspace of permutation-invariant operators in $\text{End}(\mathcal{H}_n)$ as

$$\text{End}(\mathcal{H}_n)^{S_n} := \{A \in \text{End}(\mathcal{H}_n) : \pi(A) = U_\pi A U_\pi^\dagger = A, \forall \pi \in S_n\}. \quad (2.21)$$

Clearly, this is equivalent to the set of operators that commute with all permutations. We may also call such operators simply *symmetric*, as the intended meaning of this term should always be clear from context. We now show how to construct a natural basis of the space of permutation-invariant vectors.

Definition 2.16 (Group averaging operator). Let G be a finite group with a representation on a vector space V . We define the *group averaging operator* $\mathcal{A}_G : V \rightarrow V$ by

$$\mathcal{A}_G(v) := \frac{1}{|G|} \sum_{g \in G} g(v), \quad \forall v \in V. \quad (2.22)$$

²From now on, we may sometimes refer to V as the representation of G , if the concrete action is clear from context.

Remark 2.17. This operator is sometimes called the *Reynolds operator* or *twirling operator* in the literature and is a standard tool in invariant theory. It is straightforward to verify that $\mathcal{A}_G$ is indeed a linear operator and that it projects onto the subspace of G -invariant vectors, that is, $\mathcal{A}_G(V) = V^G$.

A general construction starts from an arbitrary basis $\mathcal{B}$ of V , applies the group averaging operator to each basis vector, and then removes the resulting linear dependences ($\mathcal{A}_G(\mathcal{B})$ spans V^G). The resulting set of vectors is then a basis of V^G . However, in our case we may construct a basis of $\text{End}(\mathcal{H}_n)^{S_n}$ in a more direct way, for which we first need to introduce the notion of group orbits.

Definition 2.18 (Group orbits). Let G be a group acting on a set X . For any $x \in X$, we define the *orbit* of x under the action of G as the set

$$G \cdot x := \{g(x) : g \in G\}. \quad (2.23)$$

The set of all orbits of G on X is denoted by

$$X/G := \{G \cdot x : x \in X\}. \quad (2.24)$$

This definition allows us to rephrase the averaging operator in terms of orbits. If X is a vector space and the group action is linear, then the averaging operator may be expressed as

$$\mathcal{A}_G(x) = \frac{1}{|G|} \sum_{g \in G} g(x) = \frac{1}{|G \cdot x|} \sum_{y \in G \cdot x} y, \quad (2.25)$$

where the second equality follows from the well-known orbit-stabilizer theorem, which states that $|G| = |G \cdot x| \cdot |G_x|$, where $G_x := \{g \in G : g(x) = x\}$ is the stabilizer of x in G .

Proposition 2.19 (Orbits form equivalence classes). *Let a finite group G act on a set X . Define the relation $\sim$ on X by*

$$x \sim y \iff y = gx \quad (2.26)$$

for some $g \in G$ and all $x, y \in X$. Then $\sim$ is an equivalence relation and X is partitioned by the equivalence classes of $\sim$, which are given by the orbits of G on X .

Proof. A proof is given in [Section A.1](#). See also [\[DF03\]](#). $\square$

The following proposition shows how to construct a basis of the space of permutation-invariant operators in $\text{End}(\mathcal{H}_n)$, given a basis of $\text{End}(\mathcal{H}_n)$ that is invariant under the action of S_n .

Proposition 2.20 (Orbit basis). *Let G be a group with a representation V , and let $\mathcal{B}$ be a basis of V that is invariant under the action of G , that is, $g(\mathcal{B}) = \mathcal{B}$ for all $g \in G$. For each orbit $O \in \mathcal{B}/G$, define the unnormalized orbit sum*

$$b_O := \sum_{b \in O} b. \quad (2.27)$$

Then $\mathcal{B}^G := \{b_O : O \in \mathcal{B}/G\}$ is a basis of V^G , called the orbit basis induced by $\mathcal{B}$. In particular, $\dim V^G = |\mathcal{B}/G|$.

Proof. Each $g \in G$ permutes the elements of every orbit, so $g(b_O) = b_O$. Distinct orbits are disjoint, as seen in [Proposition 2.19](#). Hence their sums are linearly independent by linear independence of $\mathcal{B}$. Finally, if $v = \sum_{b \in \mathcal{B}} c_b b$ is invariant, comparing coefficients in $g(v) = v$ gives $c_{g(b)} = c_b$. Thus the coefficients are constant on each orbit, and $v = \sum_{O \in \mathcal{B}/G} c_O b_O$, where c_O is this common coefficient. The orbit sums therefore span V^G . $\square$

We now apply this construction to the matrix-unit basis

$$\mathcal{E}_n := \{E_{u,v} : u, v \in \{0, 1\}^n\} \quad (2.28)$$

and the Pauli basis $\mathcal{P}_n$ of $\text{End}(\mathcal{H}_n)$.

Proposition 2.21. *Both $\mathcal{E}_n$ and $\mathcal{P}_n$ are invariant as sets under the conjugation action of S_n , and therefore satisfy the assumptions of [Proposition 2.20](#). More precisely,*

$$U_\pi E_{u,v} U_\pi^\dagger = E_{\pi u, \pi v}, \quad (2.29)$$

$$U_\pi (P_1 \otimes \cdots \otimes P_n) U_\pi^\dagger = P_{\pi^{-1}(1)} \otimes \cdots \otimes P_{\pi^{-1}(n)}, \quad (2.30)$$

where $(\pi u)_j := u_{\pi^{-1}(j)}$.

Proof. Since $E_{u,v} = |u\rangle\langle v|$ and $U_\pi |u\rangle = |\pi u\rangle$, with π permuting the bitstring u , the first identity follows immediately. For arbitrary single-qubit operators $A_1, \dots, A_n$, applying $U_\pi (\bigotimes_{j=1}^n A_j) U_\pi^\dagger$ to an elementary tensor $\bigotimes_{j=1}^n v_j$ gives $\bigotimes_{j=1}^n A_{\pi^{-1}(j)} v_j$. Elementary tensors span $\mathcal{H}_n$, so

$$U_\pi \left(\bigotimes_{j=1}^n A_j \right) U_\pi^\dagger = \bigotimes_{j=1}^n A_{\pi^{-1}(j)}. \quad (2.31)$$

Taking $A_j = P_j$ proves the second identity. $\square$

The natural orbit labels record the multiplicities of the four possible tensor factors. We use the common index set

$$\Lambda_n := \{(r_1, r_2, r_3, r_4) \in \mathbb{N}^4 : r_1 + r_2 + r_3 + r_4 = n\}. \quad (2.32)$$

Definition 2.22 (Matrix-unit and Pauli orbit labels). For $u, v \in \{0, 1\}^n$, define

$$\mathbf{m}(u, v) := (m_{00}, m_{01}, m_{10}, m_{11}), \quad m_{ab} := |\{j \in \{1, \dots, n\} : (u_j, v_j) = (a, b)\}|. \quad (2.33)$$

For a Pauli string $P = \bigotimes_{j=1}^n P_j$, define

$$\mathbf{k}(P) := (k_I, k_X, k_Y, k_Z), \quad k_Q := |\{j \in \{1, \dots, n\} : P_j = Q\}|. \quad (2.34)$$

Both labels belong to Λ_n .

Proposition 2.23. *Two matrix units belong to the same S_n -orbit if and only if their labels $\mathbf{m}$ agree. Likewise, two Pauli strings belong to the same orbit if and only if their labels $\mathbf{k}$ agree.*

Proof. As $|u\rangle\langle v| = \bigotimes_{j=1}^n |u_j\rangle\langle v_j|$ (see [Equation \(2.1\)](#)), matrix units factor as $E_{u,v} = \bigotimes_{j=1}^n E_{u_j, v_j}$. By [Proposition 2.21](#), the action on either basis permutes tensor factors and therefore preserves the four counts. Conversely, if two basis elements have the same counts, ordering the tensor factors by type gives a permutation that maps one to the other. $\square$

Definition 2.24 (Matrix-unit and Pauli orbit bases). For $\mathbf{m}, \mathbf{k} \in \Lambda_n$, define the unnormalized orbit sums

$$E_{\mathbf{m}} := \sum_{\substack{u, v \in \{0, 1\}^n \\ \mathbf{m}(u, v) = \mathbf{m}}} E_{u, v}, \quad (2.35)$$

$$P_{\mathbf{k}} := \sum_{\substack{P \in \mathcal{P}_n \\ \mathbf{k}(P) = \mathbf{k}}} P. \quad (2.36)$$

By [Propositions 2.20](#) and [2.23](#), the sets

$$\mathcal{E}_n^{S_n} := \{E_{\mathbf{m}} : \mathbf{m} \in \Lambda_n\}, \quad \mathcal{P}_n^{S_n} := \{P_{\mathbf{k}} : \mathbf{k} \in \Lambda_n\} \quad (2.37)$$

are bases of $\text{End}(\mathcal{H}_n)^{S_n}$, called the *matrix-unit orbit basis* and the *Pauli orbit basis*, respectively.

With this convention, the coefficients in either orbit basis are exactly the common coefficients on the corresponding orbits in the original basis.

Corollary 2.25. *The dimension of the space of permutation-invariant operators is*

$$\dim \text{End}(\mathcal{H}_n)^{S_n} = \binom{n+3}{3} = \binom{n+3}{n} = \frac{(n+1)(n+2)(n+3)}{6} = O(n^3). \quad (2.38)$$

Proof. An elegant “stars and bars” counting argument is given in [[KLC24](#), Supplemental Material A]. $\square$

2.2.3 Schur blocks and Schur basis

Schur-Weyl duality gives a unitary decomposition

$$\mathcal{H}_n \cong \bigoplus_{k=0}^{\lfloor n/2 \rfloor} \mathcal{W}_k \otimes \mathcal{S}_k, \quad (2.39)$$

where $\mathcal{W}_k$ is the spin- $(n/2 - k)$ space and permutations act only on the factors $\mathcal{S}_k$. The dimensions are

$$d_k := \dim \mathcal{W}_k = n - 2k + 1, \quad \mu_k := \dim \mathcal{S}_k = \binom{n}{k} - \binom{n}{k-1}, \quad (2.40)$$

with $\binom{n}{-1} := 0$; see [[Sag91](#); [Ans+23](#)]. Some representation-theoretic background is also given in [Section A.4](#).

Corollary 2.26 (Schur blocks of invariant operators). *Under the isomorphism in [Equation \(2.39\)](#), an operator $A \in \text{End}(\mathcal{H}_n)$ is permutation-invariant if and only if*

$$A \cong \bigoplus_{k=0}^{\lfloor n/2 \rfloor} H_k \otimes I_{\mu_k}, \quad H_k \in \text{End}(\mathcal{W}_k). \quad (2.41)$$

The operators H_k are uniquely determined and are called the Schur blocks of A . In particular, this gives a $$ -algebra isomorphism*

$$\text{End}(\mathcal{H}_n)^{S_n} \cong \bigoplus_{k=0}^{\lfloor n/2 \rfloor} \text{End}(\mathcal{W}_k), \quad A \mapsto \bigoplus_k H_k. \quad (2.42)$$

The proof is given in [Section A.4](#). Thus an invariant operator is specified by one $d_k \times d_k$ matrix for each k , repeated μ_k times.

We finally fix the basis notation associated with this decomposition.

Definition 2.27 (Schur basis). Fix a unitary isomorphism as in [Equation \(2.39\)](#). For each k , choose an orthonormal basis $\{|k, r\rangle : k \leq r \leq n - k\}$ of $\mathcal{W}_k$ and an orthonormal basis $\{|a\rangle_k : 1 \leq a \leq \mu_k\}$ of $\mathcal{S}_k$. The corresponding vectors

$$|k, r, a\rangle := |k, r\rangle \otimes |a\rangle_k, \quad 0 \leq k \leq \lfloor n/2 \rfloor, \quad k \leq r \leq n - k, \quad 1 \leq a \leq \mu_k, \quad (2.43)$$

form an orthonormal basis of $\mathcal{H}_n$, called the *Schur basis*. The weight index r means that $|k, r, a\rangle$ is a linear combination of computational basis vectors of total Hamming weight r . For one representative copy of $\mathcal{W}_k$, we write $|u_{k,r}\rangle := |k, r, 1\rangle$.

The concrete choice of the weight vectors $|u_{k,r}\rangle$ is fixed in [Chapter 5](#) when deriving the basis transform. In these coordinates, [Equation \(2.41\)](#) gives the block entries as

$$(H_k)_{r,s} = \langle u_{k,r} | A | u_{k,s} \rangle, \quad k \leq r, s \leq n - k. \quad (2.44)$$

We have now obtained three descriptions of the same algebra $\text{End}(\mathcal{H}_n)^{S_n}$: the computational orbit basis, the Pauli orbit basis, and the Schur blocks. The first two record common coefficients on orbits, while the third makes multiplication and eigenvalue decomposition accessible through the distinct, small blocks. In the following chapters, we will study efficient transformations between these descriptions.

Chapter 3

Existing approaches and related work

We compare existing approaches to Pauli decomposition and conversion into the Schur basis introduced in [Section 2.2.3](#), focusing on their input assumptions, outputs and complexities.

Remark 3.1 (Unit-cost arithmetic assumption). Unless stated otherwise, we use a **unit-cost arithmetic model**, in which addition, subtraction, multiplication, division and square roots over $\mathbb{C}$ each have constant cost. The resulting bounds count arithmetic operations. They do not account for variable precision or guarantee uniform floating-point accuracy as n increases.

Additionally, we will use the following notation:

Definition 3.2 (Big O notation). Let $f, g : \mathbb{N} \rightarrow \mathbb{R}$ be two functions. We say that $f(n) = O(g(n))$ if there exist constants $C > 0$ and $n_0 \in \mathbb{N}$ such that for all $n \geq n_0$ we have $|f(n)| \leq C|g(n)|$.

We write $f(n) = \Theta(g(n))$ if there exist constants $C_1, C_2 > 0$ and $n_0 \in \mathbb{N}$ such that for all $n \geq n_0$ we have $C_1|g(n)| \leq |f(n)| \leq C_2|g(n)|$, and $f(n) = \Omega(g(n))$ if there exist constants $C > 0$ and $n_0 \in \mathbb{N}$ such that for all $n \geq n_0$ we have $|f(n)| \geq C|g(n)|$.

Tildes are used to suppress logarithmic factors.

3.1 General and sparse Pauli decomposition

3.1.1 Pauli decomposition of arbitrary matrices using the fast Walsh-Hadamard transform

This section summarizes the approach developed in [\[Geo+25a\]](#), where the following general formulation is studied:

Problem 3.3. Given a matrix $A \in \text{End}(\mathcal{H}_n)$, find the coefficients $\alpha(P)$ such that

$$A = \sum_{P \in \mathcal{P}_n} \alpha(P) P. \quad (3.1)$$

The access and output models, with respect to which we need to analyze algorithmic complexities, are particularly clear in this case: we are given access to a table of computational-basis matrix entries $a_{i,j}$ and aim to calculate all 4^n Pauli coefficients. We will see in later sections that these models, although very natural, are not always appropriate.

The solution given in [\[Geo+25a\]](#) utilizes the Pauli string representation in [Proposition 2.6](#) as well as the *fast Walsh-Hadamard transform* (see [\[YH96\]](#) for an introduction) to achieve a

complexity of $O(n4^n)$. Their algorithm transforms the coefficients in place, allowing for a constant memory overhead.

They arrive at their result by first deriving the following explicit formula for the Pauli coefficients [Geo+25a, Theorem 1], which we have translated into the notation introduced in Section 2.1.2:

$$\alpha_{x,z} = \frac{i^{-|x \wedge z|}}{2^n} \sum_{y \in \{0,1\}^n} A_{y \oplus x, y} (H^{\otimes n})_{y,z} \quad x, z \in \{0,1\}^n, \quad (3.2)$$

where $a_{i,j} = \langle i | A | j \rangle$ and H denotes the *Hadamard matrix*,

$$H := \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}. \quad (3.3)$$

The main contributions of [Geo+25a] are the derivation of Equation (3.2) and its combination with the Walsh-Hadamard transform to achieve the stated runtime complexity of $O(n4^n)$. In particular, the fact that only the computational-basis matrix entries along the “XOR diagonal”, $A_{y \oplus x, y}$, contribute to $\alpha_{x,z}$ is a key insight, which is also used in [SBG26a]. It should be noted that the XOR transform required to regroup computational-basis matrix entries in Equation (3.2) can also be implemented in $O(4^n)$ operations. This approach is close to optimal for the described problem, as the desired output already has size 4^n and any algorithm thus has a runtime lower bound of $\Omega(4^n)$. As the fast Walsh-Hadamard transform can be implemented in place with $O(1)$ additional memory [YH96, Chapter 7], the algorithm is also optimal in terms of memory usage. The authors provide an implementation of their algorithm at [Geo+24]. It should also be mentioned that while outputting the explicit Pauli strings with their coefficients is not included in the original formulation of the problem, this additional requirement does not change the asymptotic complexity of their algorithm.

3.1.2 Pauli decomposition of sparse matrices

While [Geo+25a] provides a solution that is close to optimal for the general problem of Pauli decomposition, it does not take into account any potential structure of the input matrix. The general algorithm also does not automatically become efficient under the following assumption of Pauli sparsity; see the cost comparison in Table 3.1. This section summarizes the recent approach developed in [SBG26a], which provides a $\text{poly}(n)$ -time classical randomized algorithm for the Pauli decomposition of matrices that are assumed to be sparse in the Pauli basis, that is, supported on at most $k = \text{poly}(n)$ Pauli strings.

Problem 3.4 (Sparse Pauli decomposition). Let $A \in \text{End}(\mathcal{H}_n)$ be a matrix promised to be equal to a linear combination of k Pauli strings:

$$A = \sum_{i=1}^k \alpha_i P_i, \quad P_i \in \mathcal{P}_n, \alpha_i \in \mathbb{C}. \quad (3.4)$$

Give an algorithm that returns all active Pauli strings P_i and their coefficients α_i .

Spencer et al. make two main assumptions: an upper bound on the sparsity k and a *sparse query access model* for the matrix A , in which they require constant-time access to all nonzero entries of A in any given row (or column).

Under these assumptions, they provide a classical algorithm that recovers the decomposition, for a given maximum failure probability δ , in $O(n^2 k^2 \log(\frac{k}{\delta}) + nk^3 \log^2(\frac{k}{\delta}))$ time, which is therefore $\text{poly}(n)$ for $k = \text{poly}(n)$.

Building on Equation (3.2), Spencer et al. exploit the fact that, for each fixed x , the coefficients $(\alpha_{x,z})_z$ are determined by the Walsh-Hadamard transform of the corresponding XOR-diagonal of A . Under the assumption of Pauli sparsity, this becomes a sparse Fourier recovery problem. Their algorithm first identifies the active x -labels through random sparse row queries. It then recovers the associated z -labels and coefficients. For each active x , the algorithm first tests whether exactly one coefficient $\alpha_{x,z}$ is nonzero. In that case, the ratios $A_{x \oplus e_j, e_j} / A_{x, 0} = (-1)^{z_j}$ determine the bits of z , where e_j is the bitstring with a single 1 in position j , and $\alpha_{x,z} = i^{-|x \wedge z|} A_{x, 0}$ gives the coefficient. The resulting candidate is checked against further XOR-diagonal entries chosen uniformly at random. If the test rejects, smaller Walsh-Hadamard transforms group the coefficients according to random linear projections of their z -labels. A coefficient that is alone in its group is recovered from the transformed values and their sign changes under shifted queries, then subtracted before the procedure is repeated [SBG26a, Sections III A–B].

The bounds on the necessary numbers of queries in their algorithm rely on the *classical uncertainty inequality for finite abelian groups* [Mes06]. Applied to $\mathbb{Z}_2^n$ ($\{0, 1\}^n$ with $\oplus$ as the group operation), it states that every nonzero function $f : \mathbb{Z}_2^n \rightarrow \mathbb{C}$ satisfies

$$|\text{supp}(f)| |\text{supp}(\widehat{f})| \geq 2^n, \quad (3.5)$$

where $\widehat{f}$ denotes its Walsh-Hadamard transform and $\text{supp}(f) = \{y \in \mathbb{Z}_2^n : f(y) \neq 0\}$.

To apply this to the discovery of active x -labels, fix such an x and define

$$b_x(y) = A_{y \oplus x, y}, \quad \beta_x(z) = \alpha_{x, z}. \quad (3.6)$$

The normalization and phase factors in Equation (3.2) do not change the support, so $\widehat{b}_x$ and β_x have the same support. The uncertainty inequality therefore gives

$$|\text{supp}(b_x)| |\text{supp}(\beta_x)| \geq 2^n. \quad (3.7)$$

Since $|\text{supp}(\beta_x)| \leq k$, at least a fraction $1/k$ of the entries of b_x are nonzero. A uniformly sampled row therefore reveals any fixed active x -label with probability at least $1/k$. Since at most k labels are active, a union bound shows that $\lceil k \ln(3k/\delta) \rceil$ independent row queries find them all with probability at least $1 - \delta/3$ [SBG26a, Lemma 4]. Similar arguments are applied for the remaining steps of the algorithm. An implementation of the algorithm is provided at [SBG26b]. It is important to note that some of the upper bounds used in their complexity analysis are not tight, and the authors themselves note that the algorithm is likely to perform better in practice than the theoretical bounds suggest. Improvements to these results are desirable. Furthermore, it may be beneficial to find a deterministic polynomial-time algorithm for the sparse Pauli decomposition problem.

Possible applications of the given algorithm include the simulation of few-body Hamiltonians [LC19, Introduction] or scenarios where the system of interest may provide a natural sparse query access model.

Remark 3.5 (Possible improvements through sparse Fourier reconstruction). The Walsh-Hadamard formulation of Spencer et al. identifies the recovery of Pauli coefficients with sparse Fourier reconstruction on $\mathbb{Z}_2^n$ (once the active x values are known), as described above. This suggests investigating whether the sparse Fourier methods discussed in Arnold's PhD thesis on sparse polynomial interpolation and testing [Arn16, Section 7.2] could improve the efficiency of their algorithm. It remains open whether these methods improve the algorithm under the same sparse query access model.

3.2 Methods for permutation-invariant operators

After dealing with the general case as well as the promise of sparse Pauli decompositions, we now turn to the problem of representing permutation-invariant operators in the Schur basis, which is the most natural basis for further computations. For simplicity, we focus on the qubit case and on the conversion from matrix-unit and Pauli orbits to Schur blocks. We will discuss several existing approaches, which utilize different ideas and are based on different assumptions. The approaches by Anschuetz et al. and Chang et al. start from Pauli orbit coordinates, while Bergh and Parentin consider matrix-unit orbits. [Chapters 4 and 5](#) derive our results on these conversions, and we will therefore compare our results to these existing approaches in [Section 3.3](#).

3.2.1 Schur block construction via tensor networks

This section summarizes [\[Ans+23\]](#), where the authors provide a method to construct the Schur representation of permutation-invariant operators using tensor networks. The problem they solve may be formulated as follows:

Problem 3.6 (Schur decomposition of permutation-invariant matrices). Given the coefficients $\alpha_{\mathbf{k}}$ of a matrix $A \in \text{End}(\mathcal{H}_n)^{S_n}$ such that

$$A = \sum_{\mathbf{k} \in \Lambda_n} \alpha_{\mathbf{k}} P_{\mathbf{k}}, \quad (3.8)$$

construct the reduced Schur basis representation of A , i.e. the distinct blocks H_k in [Equation \(2.44\)](#) for a fixed multiplicity label.

The approach by Anschuetz et al. starts with the goal of calculating all matrix entries of the Pauli orbit basis elements in the Schur basis, a natural approach for such conversion algorithms, as seen in elementary linear algebra. They provide two solutions for this problem:

- A combinatorial approach that calculates $\langle k, r, a_0 | P_{\mathbf{k}} | k, r', a_0 \rangle$ for a fixed multiplicity label a_0 and all other labels r, r' . The combinatorial formulas in [\[Ans+23, Appendix E\]](#) give an $O(n^{10})$ arithmetic-operation bound for this computation. For operators whose Pauli weight is bounded independently of n , only a constant (with respect to n) number of Pauli orbits can contribute. Using an $O(n^4)$ bound per entry of such a Pauli string in the Schur basis and the $O(n^3)$ entries across all Schur blocks for a given Pauli string, the authors obtain an $O(n^7)$ preparation bound in this case [\[Ans+23, Appendix E\]](#).
- The second approach uses the tensor network formalism, a full introduction to which would go beyond the scope of this thesis. We refer to [\[BB17\]](#) for an introduction and to [\[Ans+23, Appendix C\]](#) for a detailed description of how the tensor network formalism yields an efficient algorithm for the calculation of the necessary matrix entries. This preparation requires $O(n^7)$ arithmetic operations, again assuming unit-cost arithmetic.

A public version of their combinatorial approach is available at [\[ABK23\]](#). After computing the matrix entries, transforming any permutation-invariant operator from the symmetrized Pauli to the Schur basis is straightforward, with a general upper bound of $O(n^6)$ additional arithmetic operations [\[Ans+23, Lemma 2\]](#). As the preprocessing step depends only on n , the results (the Pauli orbits expressed in the Schur basis) may be stored and reused for multiple conversions.

For fixed-weight inputs, fewer orbit images contribute to each conversion. We discuss this reduction and the further benefit of sparse orbit-image preparation in [Section C.2.4](#). Their paper also highlights the applications of these results:

- Given ρ in Schur coordinates with the multiplicity spaces traced out, they estimate expectation values of symmetric observables in $O(n^7)$ time. The state need not be symmetric, since $\text{Tr}(\rho A) = \text{Tr}(\mathcal{A}_{S_n}(\rho) A)$ for every symmetric observable A .
- They additionally consider the case where ρ is given as a physical quantum state rather than as a classical description of its density matrix. They use the quantum Schur transform [\[BCH05\]](#) to reduce ρ to a $\lceil 2\log_2(n) \rceil$ -qubit state $\tilde{\rho}$ by discarding the multiplicity registers. Afterwards they use *classical shadow methods* developed by Huang et al. [\[HKP20\]](#) to estimate the expectation value of the observable. This requires an efficient classical description of the observable's Schur blocks [\[Ans+23\]](#), which is precisely what their results provide. Combining the Schur transform and the classical shadow method yields a sample complexity of $\tilde{O}(\|A\|_\infty^2 n^4 \varepsilon^{-2} \log \frac{1}{\delta})$ on a quantum computer of depth $\tilde{O}(n \log^m(n))$ for the estimation of the expectation value of a symmetric observable A with respect to a quantum state ρ . Here $\|A\|_\infty$ denotes the operator norm of A , ε the maximum allowed error and δ the maximum failure probability. The sample complexity may be improved, as shown in [\[CLC26, Theorem 1, Lemmas 1 and 2\]](#), essentially replacing the asymptotic factor n^4 by n^2 .
- Estimating the ground state and ground-state energy of symmetric Hamiltonians in $O(n^7)$ time. This is possible because, after the classical Schur transformation, all $O(n)$ distinct blocks, each of size at most $d \times d$ with $d = O(n)$, can be diagonalized separately. This step requires $O(n^{\tau+1})$ arithmetic operations, where τ is the matrix multiplication exponent of the implemented algorithm, with $\tau = 3$ for a naive implementation. The bottleneck therefore still lies in the Pauli-to-Schur conversion, yielding an overall complexity of $O(n^7)$.
- They also apply their results to training S_n -equivariant models in quantum machine learning [\[Ans+23, Corollary 12\]](#). Related work is discussed in [\[Sch+24\]](#).

We improve on their general conversion bound by providing an $O(n^4)$ algorithm for the Pauli-to-Schur conversion, including preprocessing, which accordingly improves the above applications.

3.2.2 Direct block construction for fixed-locality generators

We now further specialize to the case of invariant operators generated by a set of *constant locality*, an assumption that often holds in physical systems [\[LMG65; AL18\]](#). This section summarizes recent results by Chang et al. [\[CLC26\]](#). We want to reiterate that this assumption of fixed locality is different from the assumption of Pauli sparsity discussed in [Section 3.1.2](#), although fixed locality also implies polynomial Pauli sparsity [\[CLC26\]](#).

While they discuss this setting in greater detail, the most important result for our purposes is the following theorem.

Theorem 3.7 (Efficient construction of Schur blocks, [\[CLC26, Theorem 2 and Appendix A\]](#)). *Fix $k \in \mathbb{N}$ independently of n and let P be an n -qubit Pauli string satisfying either $\text{wt}(P) \leq k$ or $P = \sigma^{\otimes n}$ for some $\sigma \in \{X, Y, Z\}$. Then all nonzero entries of one copy of each Schur block of $\mathcal{A}_{S_n}(P)$ in the Schur basis can be computed classically using $O(n^2)$ arithmetic operations.*

Their algorithm improves on both the approach presented in [Ans+23] and our results in its specialized setting. The additional requirement of constant locality further reduces the number of independent degrees of freedom. Concretely, for a fixed maximum locality k and $n \geq k$, a permutation-invariant matrix has only $\binom{k+3}{k} = O(1)$ independent degrees of freedom. Still, the total number of non-zero entries in their Schur blocks is $O(n^2)$, the reason for which is that, essentially, there is a very natural relation of the fixed-weight assumption to the weight of the Schur basis vectors, as discussed in Chapter 5. Concretely, a Pauli operator of weight at most ℓ changes the Hamming weight of a given vector by at most ℓ . Because the Schur basis vectors have different Hamming weights, this forces many of the block entries to be zero. The algorithm by Chang et al. evaluates explicit formulas with a constant number of terms per entry to achieve the stated complexity of $O(n^2)$, which is optimal in terms of the number of explicitly output non-zero entries in the Schur blocks.

They go on to use these results to improve the complexity bounds of [Ans+23] for the estimation of expectation values under the assumption of constant locality of the generators. The block construction can also be used to improve the computation of ground-state energies of permutation-invariant Hamiltonians of constant locality. For exact complexity and sample bounds for their simulation framework, we refer to [CLC26, Theorem 1, Lemma 1, Lemma 2] and Section 3.3. In their paper, they also improve the classical shadow bounds used by [Ans+23] by applying classical shadow measurements and the bounds provided by [HKP20] within individual Schur sectors.

Remark 3.8. As an alternative to classical shadows, one could use full-state tomography to estimate the symmetry-reduced quantum state $\tilde{\rho}$ to trace-distance accuracy ϵ , using $\tilde{O}(n^4/\epsilon^2)$ copies at a fixed success probability [Haa+17]. This matches the sample bound in [Ans+23] up to logarithmic factors for observables of bounded spectral norm. One way to achieve this is through collective tomography using global, entangled measurements across copies, which are not always feasible in practice. This general tomography bound does not, however, match the improved sample bound of [CLC26, Lemma 1] for estimating a prescribed set of expectation values.

3.2.3 Recent results by Bergh and Parentin and prior work by Muñoz et al.

In the later stages of this thesis, we also became aware of recent work on the topic of the matrix-unit orbit to Schur block conversion by Bergh and Parentin [BP26c]. They exploit permutation invariance in a similar fashion to us, by using the results of Gijswijt [Gij09] for constructing a $\text{poly}(n)$ matrix-unit orbit to Schur block conversion algorithm. They do not, however, provide a more explicit complexity analysis of their algorithm, such as the determination of the polynomial degree.

A very interesting application of their result is provided in [BP26c, Section 4], where they use their results to compute efficient lower bounds on the channel fidelity over n parallel uses of a quantum channel. They also provide a Python package implementing their algorithm, together with many applications, at [BP26a].

After independently deriving our conversion results, we became aware of the earlier work of Muñoz et al. [Muñ+18]. In the context of collective tomography, they relate matrix-unit and Pauli orbit coordinates through hypergeometric functions [Muñ+18, Eqs. (41) and (50)], which can in turn be related to the Krawtchouk polynomials used here. They do not analyze the arithmetic complexity of this conversion. Further work is needed to turn their formulation into the bidirectional matrix-product algorithms with explicit preprocessing and complexity bounds developed in Chapter 4.

3.3 Comparison of existing approaches and our results

We first compare existing approaches with our conversion algorithms, developed in [Chapters 4 and 5](#). [Table 3.1](#) summarizes their input requirements, outputs and asymptotic complexities. For the numerical comparisons, we use an optimized implementation of the combinatorial construction of Anschuetz et al. [[Ans+23](#), Appendix E]. Implementation details, the benchmark setup and comparisons with the original public code are given in [Section C.2](#), while the fixed-locality comparisons are collected in [Section C.4](#). Our conversion algorithms and comparison implementations are available in the accompanying GitHub repository [[Sau26](#)]. Cached timings exclude preparation and measure conversion using previously prepared data.

For general Pauli orbit inputs, our benchmarks show a growing runtime advantage over the optimized combinatorial implementation of Anschuetz et al. ([Figure 3.1](#)).

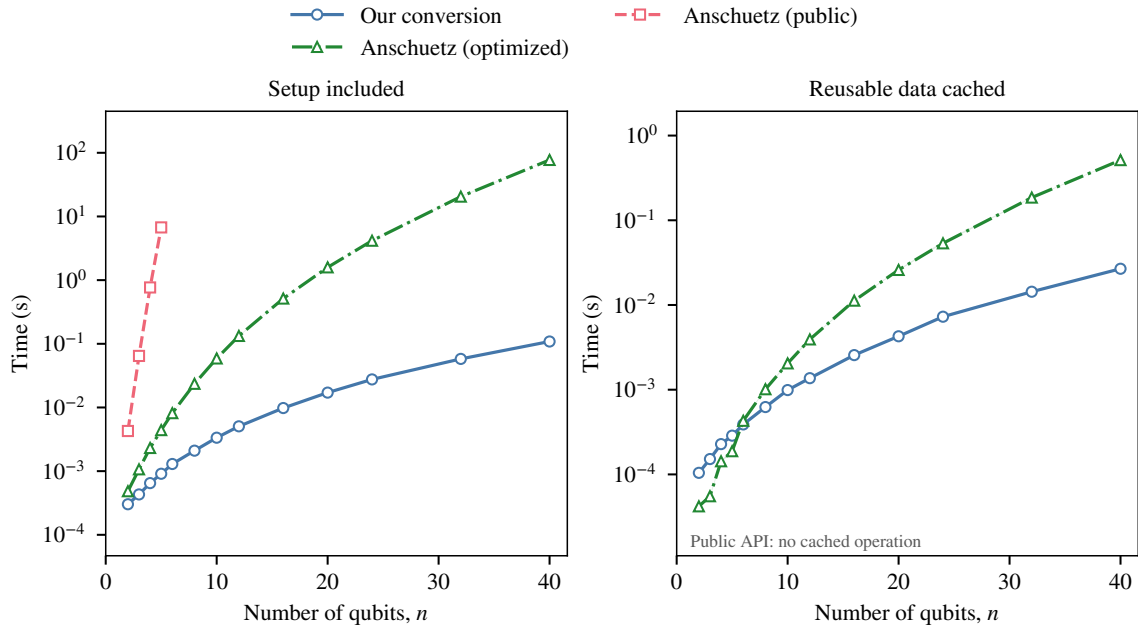


Figure 3.1: General Pauli-to-Schur conversion, including the public implementation. Our conversion uses the Hahn implementation described in [Remark 5.5](#). Left: Including preparation. Right: conversion with prepared tables. Markers and shading show medians and interquartile ranges.

Under the fixed Pauli weight assumption, first-use runtimes depend strongly on the chosen weight ([Figure C.1](#)). This comparison includes all preparation needed to convert one operator. For repeated conversions, we prepare the same Schur basis entries with each method ([Figure C.2](#)). Subsequent conversions then reuse these results. The preparation costs and the trade-off for repeated inputs are discussed further in [Section C.2.4](#).

For the matrix-unit orbit to Schur conversion, we use two implementations described in [Chapter 5](#). The shared-factor implementation uses a factorization into matrix products, while the Hahn implementation constructs the conversion matrices using a recurrence for Hahn polynomials. [Figure 3.2](#) compares our implementations with Bergh and Parentin’s `permqit` package [[BP26b](#)]. Our shared-factor implementation is faster when preprocessing is included, while cached `permqit` conversion is comparable to our Hahn implementation over the tested range. Such comparisons should, however, be treated with care, as we have not fully investigated all implementation details of the `permqit` package. Also, while we

did try to implement all algorithms as fairly as possible, we cannot guarantee complete optimization. We believe that the analysis of the concrete implementation details is a necessary next step for the analysis of the problem.

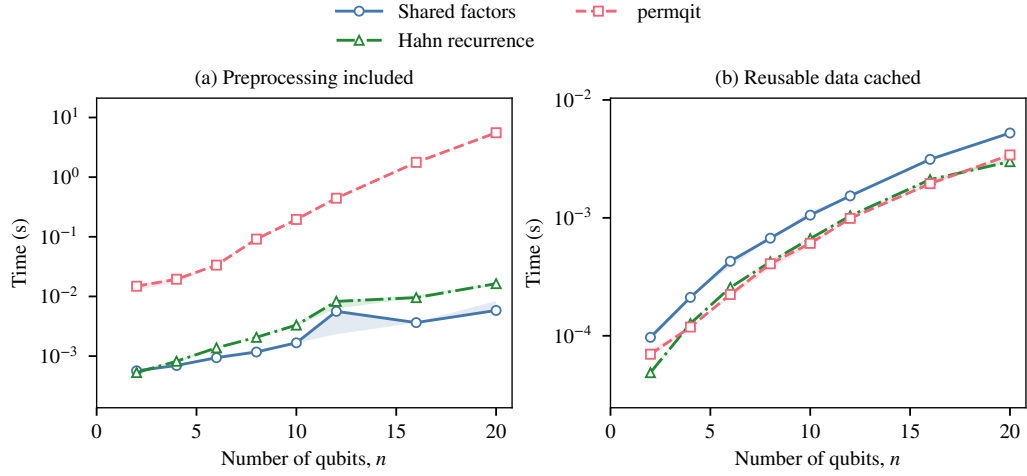


Figure 3.2: Matrix-unit orbit coefficients to all representative Schur blocks. Left: Including preparation. Right: conversion with prepared factors and transformation matrices. Markers and shading show medians and interquartile ranges over five inputs.

Table 3.1: Overview of assumptions and theoretical costs of the cited constructions. Arithmetic is assumed to have unit cost.

Approach	Input and assumptions	Output	Cost
Georges et al. [Geo+25a]	An arbitrary $2^n \times 2^n$ matrix.	All 4^n Pauli coefficients.	$O(n4^n)$ arithmetic
Spencer et al. [SBG26a]	Known bound K on Pauli sparsity, row oracle promise	Active strings with their non-zero coefficients with failure probability $< \delta$.	Runtime bound T . Query bound Q .
Anschuetz et al. (tensor network) [Ans+23]	An invariant operator as Pauli-orbit coefficients.	Distinct Schur blocks.	$O(n^7)$ preparation, $O(n^6)$ per conversion.
Chang et al. [CLC26]	One fixed weight orbit average or $X^{\otimes n}$, $Y^{\otimes n}$, $Z^{\otimes n}$.	Nonzero Schur entries.	$O(n^2)$
Bergh and Parentin [BP26c]	An invariant operator as matrix-unit orbit coefficients.	Distinct Schur blocks.	$\text{poly}(n)$, no explicit complexity analysis.
This work	General invariant operator as Pauli orbit, matrix-unit orbit coefficients or distinct Schur blocks.	Any of the three compressed descriptions.	$O(n^4)$ using naive, $O(n^{1+\tau})$ using fast matrix multiplication.

$$T = O(n^2 K^2 \log(K/\delta) + nK^3 \log(K/\delta)^2), \quad Q = O(nK^2 \log(K/\delta) + K^3 \log(K/\delta)^2).$$

Here $K = \text{poly}(n)$. The runtime bound T charges $O(n)$ for operations on n -bit labels. The query bound Q counts each retrieval of an entire sparse row as one query, in accordance with Spencer et al.'s access model.

Chapter 4

Matrix-unit orbits and Pauli orbits

In this chapter we use prior results regarding the action of the Hadamard matrix on permutation-invariant operators to derive an efficient, bidirectional transformation between the computational orbit basis and the Pauli orbit basis. We then analyze the complexity of this transformation. These aspects make up the main contributions of our work and have, as shown in [Chapter 6](#), direct applications in simulating quantum systems.

4.1 Deriving the efficient transformation

Problem 4.1. Given an invariant $A \in \text{End}(\mathcal{H}_n)^{S_n}$, convert between the representations:

$$A = \sum_{m \in \Lambda_n} a_m E_m = \sum_{k \in \Lambda_n} \alpha_k P_k, \quad (4.1)$$

where the E_m and P_k are the matrix-unit and Pauli orbit bases, as introduced in [Section 2.2.2](#). The given input is thus either the coefficients a_m or α_k , and the output is the other set of coefficients.

With classical query access to the computational-basis entries of the invariant operator A , each a_m can be read from any entry $\langle u|A|v \rangle$ with $\mathbf{m}(u, v) = \mathbf{m}$. Thus $\binom{n+3}{3} = O(n^3)$ entry queries supply the full matrix-unit input.

4.1.1 The one-qubit change of basis

We first derive the one-qubit (that is, $n = 1$) solution to [Problem 4.1](#). In this case the S_n action is trivial and the orbit sums are exactly the original matrix units and Pauli matrices. An easy calculation proves the following four identities:

$$\begin{aligned} E_{00} &= (I + Z)/2, & E_{11} &= (I - Z)/2, \\ E_{10} &= (X - iY)/2, & E_{01} &= (X + iY)/2. \end{aligned} \quad (4.2)$$

Consequently, the Hilbert space $\text{End}(\mathcal{H}_1)$ of one-qubit operators decomposes as

$$\begin{aligned} \text{End}(\mathcal{H}_1) &= U \oplus W, \\ U &= \text{span}\{E_{00}, E_{11}\} = \text{span}\{I, Z\}, \\ W &= \text{span}\{E_{10}, E_{01}\} = \text{span}\{X, Y\}. \end{aligned} \quad (4.3)$$

For this local calculation we reorder the bases as $(E_{00}, E_{11}, E_{10}, E_{01})$ and (I, Z, X, Y) . The orbit labels retain the orders $(E_{00}, E_{01}, E_{10}, E_{11})$ and (I, X, Y, Z) from [Section 2.2.2](#). The

change of basis from matrix units to the Pauli basis is given by the matrix

$$F = \frac{1}{2}(H \oplus DH), \quad H = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad D = \text{diag}(1, -i). \quad (4.4)$$

By taking tensor products, we see that the change of basis from matrix units to the Pauli basis for the (non-symmetric) n -qubit space $\text{End}(\mathcal{H}_n)$ is given by $F^{\otimes n}$.

4.1.2 Restriction to symmetric tensors

Instead of constructing $F^{\otimes n}$ explicitly, we consider the induced change of coordinates on permutation-invariant operators. Following [FK07, Section 3], we describe this restriction using symmetric tensor powers.

For a finite-dimensional complex vector space V , we use the invariant-tensor realization of its m th symmetric power,

$$\text{Sym}^m(V) := (V^{\otimes m})^{S_m}, \quad (4.5)$$

with $\text{Sym}^0(V) = \mathbb{C}$. For a more abstract introduction to the space $\text{Sym}^m(V)$, see [FH13, Appendix B] or [Bou98, Chapter 3]. Since applying the same linear map $T : V \rightarrow V$ to every tensor factor commutes with permutations, $T^{\otimes m}$ induces the map

$$\text{Sym}^m(T) := T^{\otimes m}|_{\text{Sym}^m(V)}. \quad (4.6)$$

This construction preserves composition, as shown in [FK07, Section 3]. Taking $V = \text{End}(\mathcal{H}_1)$, the identification $\text{End}(\mathcal{H}_n) \cong V^{\otimes n}$ gives

$$\text{End}(\mathcal{H}_n)^{S_n} \cong \text{Sym}^n(V). \quad (4.7)$$

Furthermore, let $J_E, J_P : \mathbb{C}^4 \rightarrow V$ be the coordinate isomorphisms associated with the ordered, one-qubit matrix-unit and Pauli bases. Since $J_E = J_P \circ F$, taking tensor powers and restricting to symmetric tensors gives

$$\text{Sym}^n(J_E) = \text{Sym}^n(J_P) \circ \text{Sym}^n(F). \quad (4.8)$$

The tensor powers $J_E^{\otimes n}$ and $J_P^{\otimes n}$ replace each standard basis vector of $\mathbb{C}^4$ in a tensor product by its corresponding matrix unit or Pauli matrix, respectively. Since these maps send distinct basis tensors to distinct basis tensors and commute with permutations, they send each unnormalized orbit sum to the corresponding unnormalized operator orbit sum.

For example, let e_1, e_2 be the first two standard basis vectors of $\mathbb{C}^4$. With our chosen local basis orders,

$$\begin{aligned} J_E^{\otimes 2}(e_1 \otimes e_2 + e_2 \otimes e_1) &= E_{00} \otimes E_{11} + E_{11} \otimes E_{00}, \\ J_P^{\otimes 2}(e_1 \otimes e_2 + e_2 \otimes e_1) &= I \otimes Z + Z \otimes I. \end{aligned} \quad (4.9)$$

The right-hand sides are the respective operator orbit sums, with every distinct term occurring exactly once. Consequently, the matrix of $\text{Sym}^n(F)$ in the standard orbit-sum basis is the transition matrix between the coefficients in the matrix-unit and Pauli orbit bases.

The decomposition $V = U \oplus W$ yields, as shown in [Bou98, Chapter 3, Section 6, Proposition 9], a canonical decomposition of the symmetric power:

$$\text{Sym}^n(V) \cong \bigoplus_{w=0}^n \text{Sym}^{n-w}(U) \otimes \text{Sym}^w(W). \quad (4.10)$$

As we have $W = \text{span}\{E_{10}, E_{01}\} = \text{span}\{X, Y\}$, the index w counts off-diagonal matrix-unit factors, or equivalently X and Y factors:

$$w = m_{01} + m_{10} = k_X + k_Y. \quad (4.11)$$

Since $F = \frac{1}{2}(H \oplus DH)$, its induced transition matrix on the w th summand is

$$2^{-n} \text{Sym}^{n-w}(H) \otimes \text{Sym}^w(DH). \quad (4.12)$$

We again refer to [Bou98, Chapter 3] for further details.

4.1.3 The separated Krawtchouk transform

We now express the symmetric-power maps obtained above in the orbit-sum bases. The symmetric powers of H are represented by Krawtchouk matrices, while the diagonal factor D contributes a phase. This yields the following theorem, the main result of this chapter.

Theorem 4.2 (Separated Krawtchouk transform). *Let $A \in \text{End}(\mathcal{H}_n)^{S_n}$, and let $M^{(w)}$ and $C^{(w)}$ be its orbit coefficient tables defined in Equation (4.18). With the Krawtchouk matrices Q_m and phase matrices D_w defined in Equations (4.17) and (4.19), the matrix-unit-to-Pauli conversion is*

$$C^{(w)} = 2^{-n} Q_{n-w} M^{(w)} Q_w^T D_w, \quad 0 \leq w \leq n. \quad (4.13)$$

Proof. Feinsilver and Kocik identify the symmetric Hadamard action with a Krawtchouk matrix [FK07, Proposition 3.1]. Here we translate this correspondence into our notation.

For an ordered basis (b_0, b_1) , to be identified later with the bases of U and W from Equation (4.3), let $s_h^{(m)}$ denote the sum of all distinct elementary tensors containing h copies of b_1 and $m - h$ copies of b_0 . Define the Krawtchouk polynomials by

$$(1+z)^{m-g}(1-z)^g = \sum_{h=0}^m K_h(g; m) z^h, \quad 0 \leq g \leq m. \quad (4.14)$$

To obtain the symmetric Hadamard action, observe that $(b_0 + zb_1)^{\otimes m} = \sum_{h=0}^m z^h s_h^{(m)}$. Since $Hb_0 = b_0 + b_1$ and $Hb_1 = b_0 - b_1$, applying H to every factor gives

$$\begin{aligned} \sum_{h=0}^m z^h H^{\otimes m} s_h^{(m)} &= ((1+z)b_0 + (1-z)b_1)^{\otimes m} \\ &= \sum_{g=0}^m (1+z)^{m-g}(1-z)^g s_g^{(m)}. \end{aligned} \quad (4.15)$$

Comparing coefficients of z^h using Equation (4.14), we obtain

$$H^{\otimes m} s_h^{(m)} = \sum_{g=0}^m K_h(g; m) s_g^{(m)}. \quad (4.16)$$

Accordingly, we write

$$[Q_m]_{g,h} := K_h(g; m), \quad 0 \leq g, h \leq m. \quad (4.17)$$

Thus Q_m is the matrix of $\text{Sym}^m(H)$ in the ordered orbit-sum basis $(s_0^{(m)}, \dots, s_m^{(m)})$.

We now arrange the orbit coefficients so that the two factors in Equation (4.12) act on separate indices. Fix w . In the input bases (E_{00}, E_{11}) and (E_{10}, E_{01}) , the counts of the

second basis vectors are $h_0 = m_{11}$ and $h_1 = m_{01}$. In the output bases (I, Z) and (X, Y) , the corresponding counts are $g_0 = k_Z$ and $g_1 = k_Y$. Define the coefficient tables

$$\begin{aligned} M_{h_0, h_1}^{(w)} &:= a_{(n-w-h_0, h_1, w-h_1, h_0)}, \\ C_{g_0, g_1}^{(w)} &:= \alpha_{(n-w-g_0, w-g_1, g_1, g_0)}. \end{aligned} \quad (4.18)$$

Both tables have shape $(n-w+1) \times (w+1)$, with $0 \leq h_0, g_0 \leq n-w$ and $0 \leq h_1, g_1 \leq w$. Thus rows record the diagonal counts and columns record the off-diagonal counts.

The remaining diagonal factor acts as $D^{\otimes w} s_{g_1}^{(w)} = (-i)^{g_1} s_{g_1}^{(w)}$. Thus, setting

$$D_w := \text{diag}((-i)^g)_{g=0}^w, \quad (4.19)$$

the matrix of $\text{Sym}^w(DH)$ is $D_w Q_w$. Consequently, Equation (4.12) becomes

$$2^{-n} Q_{n-w} \otimes (D_w Q_w). \quad (4.20)$$

Explicitly, the entries satisfy

$$C_{g_0, g_1}^{(w)} = 2^{-n} (-i)^{g_1} \sum_{h_0=0}^{n-w} \sum_{h_1=0}^w K_{h_0}(g_0; n-w) M_{h_0, h_1}^{(w)} K_{h_1}(g_1; w). \quad (4.21)$$

In matrix form, this is

$$C^{(w)} = 2^{-n} Q_{n-w} M^{(w)} (D_w Q_w)^T = 2^{-n} Q_{n-w} M^{(w)} Q_w^T D_w.$$

This proves Equation (4.13). Thus, each fixed w component separates into two Krawtchouk transforms and a diagonal phase factor. $\square$

4.1.4 The inverse transform

Since $H^2 = 2I$, the symmetric Hadamard action Q_m satisfies

$$Q_m^2 = 2^m I_{m+1}, \quad (4.22)$$

where I_{m+1} is the $(m+1) \times (m+1)$ identity matrix. Therefore, the inverse of Equation (4.13) is

$$M^{(w)} = Q_{n-w} C^{(w)} D_w^{-1} Q_w^T, \quad 0 \leq w \leq n, \quad (4.23)$$

where $D_w^{-1} = \text{diag}(i^g)_{g=0}^w$.

Proof. Multiplying Equation (4.13) on the left by Q_{n-w} and on the right by $D_w^{-1} Q_w^T$ gives

$$\begin{aligned} Q_{n-w} C^{(w)} D_w^{-1} Q_w^T &= 2^{-n} Q_{n-w}^2 M^{(w)} (Q_w^T)^2 \\ &= 2^{-n} 2^{n-w} 2^w M^{(w)} = M^{(w)}. \end{aligned}$$

$\square$

4.2 Complexity analysis

The Krawtchouk polynomials from Equation (4.14) satisfy the three-term recurrence [Col11, Theorem 4.1, with $s = 2$]

$$(h+1)K_{h+1}(g; m) = (m-2g)K_h(g; m) - (m-h+1)K_{h-1}(g; m), \quad (4.24)$$

for $0 \leq g \leq m$ and $1 \leq h < m$. The initial values are

$$K_0(g; m) = 1, \quad K_1(g; m) = m - 2g, \quad (4.25)$$

where the second value is needed only for $m \geq 1$, and $Q_0 = (1)$.

We use the unit-cost arithmetic model of Remark 3.1. The input and output are the orbit coefficients as specified in Problem 4.1.

Theorem 4.3 (Arithmetic complexity). *Given all matrix-unit orbit coefficients of $A \in \text{End}(\mathcal{H}_n)^{S_n}$, its Pauli orbit coefficients can be computed in $O(n^4)$ arithmetic operations using naive matrix multiplication. Using a faster matrix multiplication algorithm which multiplies two $n \times n$ matrices in $O(n^\tau)$ operations, the complexity can be reduced to $O(n^{1+\tau})$. The same bound holds for the inverse conversion.*

Remark 4.4. Both coordinate vectors have $\binom{n+3}{3} = \Theta(n^3)$ entries, so writing the complete output requires $\Omega(n^3)$ time. Our bounds are therefore within factors $O(n)$ and $O(n^{\tau-2})$, respectively, of this input/output lower bound.

4.2.1 Naive implementation

We first construct $[Q_m]_{g,h} := K_h(g; m)$ for $0 \leq m \leq n$ and $0 \leq g, h \leq m$. For each fixed m and g , Equation (4.25) gives the first entries, and Equation (4.24) computes the remaining entries in increasing order of h . Each step uses a bounded (independently of m, g, h !) number of arithmetic operations. Thus one row requires $O(m+1)$ operations and the full matrix Q_m requires $O((m+1)^2)$ operations. Since

$$\sum_{m=0}^n (m+1)^2 = \frac{(n+1)(n+2)(2n+3)}{6} = O(n^3), \quad (4.26)$$

all Krawtchouk matrices can be constructed in $O(n^3)$ operations.

For a fixed w , put $R_w = n - w + 1$ and $S_w = w + 1$, so that $M^{(w)}$ and $C^{(w)}$ have R_w rows and S_w columns. We evaluate Equation (4.13) by first computing $Q_{n-w}M^{(w)}$ and then multiplying the result by Q_w^T . Using naive matrix multiplication these are two products of rectangular matrices, with the first having dimensions $R_w \times R_w$ and $R_w \times S_w$, and the second having dimensions $R_w \times S_w$ and $S_w \times S_w$. Using naive matrix multiplication, the first product therefore has $R_w S_w$ entries, each requiring R_w multiplications and $R_w - 1$ additions. The second product has the same number of entries, each requiring S_w multiplications and $S_w - 1$ additions. Together, these products therefore require at most

$$2(R_w^2 S_w + R_w S_w^2) \quad (4.27)$$

arithmetic operations. Multiplying by the diagonal matrix D_w and the scalar 2^{-n} requires at most $2R_w S_w$ further multiplications.

Using $R_w + S_w = n + 2$ and summing over all w , the sum giving the entire matrix multiplication cost is

$$\begin{aligned} \sum_{w=0}^n (R_w^2 S_w + R_w S_w^2) &= (n+2) \sum_{w=0}^n R_w S_w \\ &= (n+2) \binom{n+3}{3} = O(n^4). \end{aligned} \tag{4.28}$$

The phase multiplications and scalar rescaling cost $\sum_{w=0}^n 2R_w S_w = O(n^3)$ additional operations in total. Reindexing as in [Equation \(4.18\)](#) can trivially be achieved in $O(n^3)$ operations (namely lookup and writing).

For the inverse in [Equation \(4.23\)](#), we first scale the columns of $C^{(w)}$ by D_w^{-1} and then perform the two Krawtchouk matrix multiplications. Their dimensions are identical to those in the forward direction, so the same operation count applies. Including the $O(n^3)$ preprocessing cost proves the $O(n^4)$ bound in both directions.

4.2.2 Fast matrix multiplication improvement

This bound may be improved slightly using efficient matrix multiplication algorithms.

Suppose two $d \times d$ complex matrices can be multiplied in $O(d^\tau)$ arithmetic operations for some $2 \leq \tau < 3$. Since $R_w, S_w \leq n+1$, both products for each w can be padded with zeros to square products of size $n+1$, with a cost of $O(n^2)$ per block and therefore a total cost of $O(n^3)$. Each of the $n+1$ blocks then has a multiplication cost of $O(n^\tau)$ plus an additional $O(n^2)$ cost of extracting the relevant submatrix. Thus the total cost of all $n+1$ blocks is $O(n^{1+\tau})$ operations in either direction. Strassen's (non-galactic, practical) algorithm [\[Str69\]](#) gives $\tau = \log_2 7 \approx 2.807$, while the current best-known bound is $\tau = 9/4 = 2.25$ (although the associated algorithm is galactic, i.e. inefficient for any reasonable number of qubits). For more details regarding the implementation of Strassen for rectangular matrices see [\[Bre70, Section 4\]](#). This matrix-multiplication bound does not establish floating-point accuracy for the complete coordinate conversion.

Pseudocode for both directions and remarks on the Python implementation are given in [Sections C.1.1](#) and [C.1.3](#). The Python implementation is available in [\[Sau26\]](#).

Chapter 5

Schur blocks and matrix-unit orbits

In this chapter we give a proof of an efficient conversion algorithm between the previously introduced Schur block form and the computational orbit basis of an S_n -invariant matrix. It is in large part based on prior work which studied the same setting in the context of the binary Hamming Terwilliger algebra. Our concrete implementation is largely based on works of Gijswijt [Gij09] and Vallentin [Val09]. As far as we are aware, this is the first time the conversion complexity has been analyzed explicitly.

5.1 Deriving the efficient transformation

Problem 5.1. Given an invariant $A \in \text{End } \mathcal{H}_n^{S_n}$, convert between the representations

$$A = \sum_{\mathbf{m} \in \Lambda_n} a_{\mathbf{m}} E_{\mathbf{m}} \cong \bigoplus_{k=0}^{\lfloor n/2 \rfloor} H_k \otimes I_{\mu_k}, \quad (5.1)$$

where the Schur basis (Section 2.2.3) is fixed. The input is the set of coefficients $a_{\mathbf{m}}$, the output the set of Schur blocks H_k , and vice versa.

5.1.1 Choice of coordinates

We start by relabeling the matrix-unit orbits in a way that allows a more concise description of the transformation. For binary strings $x, y \in \{0, 1\}^n$, set $r = |x|$, $s = |y|$ and $t = |x \wedge y|$. Define

$$B_{rs}^t := \sum_{\substack{x, y \in \{0, 1\}^n \\ |x|=r, |y|=s, |x \wedge y|=t}} E_{xy}, \quad t_{\min} := \max(0, r + s - n) \leq t \leq \min(r, s) =: t_{\max}. \quad (5.2)$$

This choice corresponds to the following relabeling of the matrix-unit orbits:

$$\mathbf{m} = (m_{00}, m_{01}, m_{10}, m_{11}) = (n - r - s + t, s - t, r - t, t), \quad a_{rs}^t := a_{\mathbf{m}}. \quad (5.3)$$

In accordance with Section 2.2.2, we have $v_{rs}^t := \|B_{rs}^t\|_{\text{HS}}^2 = \frac{n!}{(n-r-s+t)!(r-t)!(s-t)!t!}$. We choose the following representatives of a Schur basis as in Definition 2.27:

$$|u_{k,k}\rangle = 2^{-k/2}(|01\rangle - |10\rangle)^{\otimes k} \otimes |0\rangle^{\otimes (n-2k)}, \quad 0 \leq k \leq \lfloor n/2 \rfloor, \quad (5.4)$$

$$S_- = \sum_{j=1}^n |1\rangle\langle 0|_j, \quad |1\rangle\langle 0|_j := I^{\otimes (j-1)} \otimes |1\rangle\langle 0| \otimes I^{\otimes (n-j)} \quad (5.5)$$

$$|u_{k,r}\rangle = \sqrt{\frac{(n-k-r)!}{(r-k)!(n-2k)!}} S_-^{r-k} |u_{k,k}\rangle, \quad k \leq r \leq n-k. \quad (5.6)$$

After a permutation of tensor factors fixed for each k , these are the normalized vectors $e_{k,r}$ in Gijswijt's block diagonalization [Gij09, Theorem 6 and Section 5, Equations (53)–(54)]. We therefore take $|u_{k,r}\rangle = |k, r, 1\rangle$. Further details regarding the identification with Gijswijt's definition and some elementary Hamming weight properties are given in Section A.5.

5.1.2 The explicit transformation formula

By definition, every matrix-unit orbit B_{rs}^t maps a vector of (Hamming) weight s to a vector of weight r . Vectors of weight not equal to s are mapped to the zero vector. For a fixed k , the Schur basis vectors $|u_{k,\hat{k}}\rangle$ have weight $\hat{k}$, as verified in Section A.5. Therefore, the matrix elements of B_{rs}^t in each $\mathcal{W}_k$ -block are zero, except for

$$K_{k;rs}^t := \langle u_{k,r} | B_{rs}^t | u_{k,s} \rangle, \quad 0 \leq k \leq \kappa_{rs} := \min(r, s, n-r, n-s). \quad (5.7)$$

The following proposition gives an explicit, efficiently implementable formula for computing these matrix elements.

Proposition 5.2. *The matrix elements $K_{k;rs}^t$ are given by*

$$K_{k;rs}^t = \sqrt{\frac{\binom{n-2k}{r-k}}{\binom{n-2k}{s-k}}} \sum_{l=0}^{s-k} (-1)^{s-t-l} \binom{n-k-r}{l} \binom{r-k}{s-k-l} \binom{k}{s-t-l}, \quad (5.8)$$

where $\binom{a}{b} = 0$ if $b < 0$ or $b > a$.

Proof. This proposition follows from [Gij09, Equations (54)–(55), Section 5], after normalizing the representatives, relabeling the indices and correcting a typo that places the factor $(-1)^{s-t-l}$ outside the sum (see [Gij09, Equation (52)]). For the detailed proof we refer to [Gij09] or equivalently [Val09]. $\square$

We state the conversion result in the following theorem.

Theorem 5.3 (Matrix-unit orbits/Schur block conversion). *In the previously defined coordinates,*

$$(H_k)_{r,s} = \sum_{t=\max(0, r+s-n)}^{\min(r,s)} a_{rs}^t K_{k;rs}^t, \quad (5.9)$$

$$a_{rs}^t = \frac{1}{v_{rs}^t} \sum_{k=0}^{\min(r,s,n-r,n-s)} \mu_k (H_k)_{r,s} K_{k;rs}^t, \quad (5.10)$$

where $\mu_k = \binom{n}{k} - \binom{n}{k-1}$ is the multiplicity of the k -th Schur block.

Proof. The first equation follows trivially from linearity and the definition of $K_{k;rs}^t$. For the second equation, consider

$$v_{rs}^t a_{rs}^t = \langle B_{rs}^t, A \rangle_{\text{HS}} = \sum_k \mu_k \overline{K_{k;rs}^t} (H_k)_{r,s}. \quad (5.11)$$

As we can see in Proposition 5.2, the matrix elements $K_{k;rs}^t$ are real with our chosen convention, and the second equation follows. $\square$

5.2 Complexity analysis

We use the unit-cost arithmetic model, including division and square roots. The input and output are the coordinates specified in [Problem 5.1](#).

Theorem 5.4 (Arithmetic complexity). *The conversion between matrix-unit orbit coefficients and distinct Schur blocks can be computed in $O(n^4)$ arithmetic operations using naive matrix multiplication. If two $d \times d$ matrices can be multiplied in $O(d^\tau)$ operations for some $2 \leq \tau < 3$, the conversion requires $O(n^{1+\tau})$ operations.*

5.2.1 Naive implementation

Instead of evaluating the sum in [Proposition 5.2](#) separately for every coefficient, which would result in an $O(n^5)$ preprocessing bound, we arrange the computation so that all column weights s are processed together for each row weight r . This reduces the conversion to two matrix products per value of r .

To this end, define two $(n+1) \times (n+1)$ matrices by

$$\begin{aligned} F_{ku} &:= \frac{(n-k-u)!}{(u-k)!}, \quad 0 \leq k \leq \lfloor n/2 \rfloor, \quad k \leq u \leq n-k, \\ G_{ut} &:= (-1)^{u-t} \binom{u}{t}, \quad 0 \leq t \leq u \leq n. \end{aligned} \tag{5.12}$$

Both matrices are indexed from 0 to n , and all entries outside the stated ranges are set to zero. Since the same factors F and G are used for every pair of weights r, s , we call this the shared-factor implementation. For admissible k, r, s , also define

$$\gamma_{k;rs} := \sqrt{\frac{(r-k)!(s-k)!}{(n-k-r)!(n-k-s)!}}. \tag{5.13}$$

The matrix elements can then be written as

$$K_{k;rs}^t = \gamma_{k;rs} \sum_{u=0}^{\min(r,s)} \frac{F_{ku} G_{ut}}{(r-u)!(s-u)!}. \tag{5.14}$$

This follows by expanding the binomial coefficients in an equivalent formula of Gijswijt. The short derivation is given in [Section B.3](#). Therefore, the conversion formulas of [Theorem 5.3](#) become

$$(H_k)_{r,s} = \sum_{t=\max(0,r+s-n)}^{\min(r,s)} a_{rs}^t \gamma_{k;rs} \sum_{u=0}^{\min(r,s)} \frac{F_{ku} G_{ut}}{(r-u)!(s-u)!}, \tag{5.15}$$

$$a_{rs}^t = \frac{1}{v_{rs}^t} \sum_{k=0}^{\min(r,s,n-r,n-s)} \mu_k (H_k)_{r,s} \gamma_{k;rs} \sum_{u=0}^{\min(r,s)} \frac{F_{ku} G_{ut}}{(r-u)!(s-u)!}. \tag{5.16}$$

For the conversion from matrix-unit orbits to Schur blocks, fix r and construct the square matrix $X^{(r)}$ with rows indexed by t and columns indexed by s . Set $X_{ts}^{(r)} = a_{rs}^t$ for $0 \leq s \leq n$ and $\max(0, r+s-n) \leq t \leq \min(r, s)$ and set all other entries to zero. First compute $GX^{(r)}$. Divide its (u, s) -entry by $(r-u)!(s-u)!$ when $u \leq \min(r, s)$, and set the remaining entries to zero. Multiply the resulting matrix on the left by F . Finally, multiply each admissible (k, s) entry by $\gamma_{k;rs}$ to obtain $(H_k)_{r,s}$. These steps evaluate the forward conversion in [Equation \(5.15\)](#).

For the inverse conversion, define $X^{(r)}$ instead with rows indexed by k and columns indexed by s . Set $X_{ks}^{(r)} = \mu_k \gamma_{k;rs}(H_k)_{r,s}$ when $k \leq \kappa_{rs}$ and set all other entries to zero. Compute $F^T X^{(r)}$, apply the same division by $(r-u)!(s-u)!$ and the same zeroing of entries, and multiply the result on the left by G^T . Dividing each admissible (t, s) entry by v_{rs}^t gives the inverse conversion in Equation (5.16).

The necessary factorials $0!, \dots, n!$ can be generated successively in $O(n)$ operations, and Pascal's rule constructs all required binomial coefficients in $O(n^2)$ operations (see [Ski20, Section 10.1.4] for the explicit algorithm). Each entry of F and G then takes a constant number of operations, so both matrices cost $O(n^2)$ operations to construct. The factorial table also gives each $\gamma_{k;rs}$ and v_{rs}^t in $O(1)$. There are $O(n^3)$ such values, while the multiplicities μ_k require only $O(n)$ further operations. Thus, the total preprocessing cost is $O(n^3)$.

For each r , both of the two square matrix products have $(n+1)^2$ entries. Using naive multiplication, every entry requires $n+1$ multiplications and n additions. Summing over the $n+1$ row weights therefore gives

$$\sum_{r=0}^n 2(n+1)^2(2n+1) = O(n^4) \quad (5.17)$$

arithmetic operations for the products. The divisions and rescalings require $O(n^2)$ further operations per r , hence $O(n^3)$ in total. Reindexing the coefficients using Equation (5.3), filling the tables with zeros and extracting the output likewise require $O(n^3)$ index operations, assumed to be constant-time. This proves the $O(n^4)$ bound in both directions.

5.2.2 Fast matrix multiplication improvements

Fast matrix multiplication may again be used for improving the asymptotic complexity. Applying a square matrix multiplication algorithm with cost $O(d^\tau)$ to each of the two products per row weight r gives

$$2(n+1)O((n+1)^\tau) + O(n^3) = O(n^{1+\tau}), \quad (5.18)$$

where the $O(n^3)$ term includes preparation of the factors and weights, reindexing and entrywise scaling. Padding and extracting the desired submatrix cost $O(n^2)$ per product and therefore $O(n^3)$ over the whole conversion. The inverse uses the same number and sizes of products, which proves the remaining claim of Theorem 5.4. The factorization establishes the arithmetic bounds above under the unit-cost model. These counts do not include a precision increase to maintain a prescribed error as n grows. For most numerical comparisons and applications, we evaluate the same conversion using the Hahn recurrence below. The numerical considerations behind this choice are discussed in Sections C.2.3 and C.2.5. Pseudocode for the factorized conversion in both directions and remarks on the Python implementation are given in Sections C.1.2 and C.1.3. Python implementations of the factorial formula and the Hahn recurrence are available in [Sau26].

Remark 5.5 (Hahn recurrence implementation). Following Vallentin [Val09, Section 4], the matrix elements can also be expressed using Hahn polynomials. For $r \leq s$, write $q_k(t) = Q_k(r-t; s-n-1, -s-1, r)$. Starting with $q_{-1}(t) = 0$ and $q_0(t) = 1$, we use the degree recurrence [Nat, Equations 18.22.2–3]

$$A_k q_{k+1}(t) = (A_k + C_k - r + t)q_k(t) - C_k q_{k-1}(t), \quad 0 \leq k < \kappa_{rs}, \quad (5.19)$$

where

$$\begin{aligned} A_k &= \frac{(n+1-k)(n-s-k)(r-k)}{(n+1-2k)(n-2k)}, \\ C_k &= \frac{k(n+1-r-k)(s+1-k)}{(n+2-2k)(n+1-2k)}. \end{aligned} \quad (5.20)$$

For $r > s$, interchange the two weights. Normalizing each row $(\sqrt{v_{rs}^t} q_k(t))_t$ to unit Euclidean norm gives the orthogonal Hahn transform matrix

$$U_{kt} = \sqrt{\frac{\mu_k}{v_{rs}^t}} K_{k;r,s}^t. \quad (5.21)$$

This characterizes the exact matrix. The implementation evaluates its normalization analytically in the logarithmic domain. For $r \leq s$, put

$$\rho_0 = 1, \quad \rho_{k+1} = \rho_k \frac{(r-k)(n-s-k)}{(s-k)(n-r-k)}, \quad U_{kt} = \sqrt{\frac{\mu_k v_{rs}^t \rho_k}{\binom{n}{r} \binom{n}{s}}} q_k(t). \quad (5.22)$$

The recurrence for ρ_k is used only for $k < \kappa_{rs}$. The factors follow by evaluating [Proposition 5.2](#) at $t = r$ and using $q_k(r) = 1$. The positive square root gives the signs of the representatives in [Equation \(5.6\)](#). Thus the code does not normalize the computed rows by their measured Euclidean norms. It combines logarithms of the positive factors with $\log |q_k(t)|$ and retains the sign of $q_k(t)$, with zero polynomial entries giving zero matrix entries. With $x_t = \sqrt{v_{rs}^t} a_{rs}^t$ and $y_k = \sqrt{\mu_k} (H_k)_{r,s}$, the two conversions are $y = Ux$ and $x = U^T y$.

For fixed r, s , both k and t have $\kappa_{rs} + 1$ admissible values, so U is a $(\kappa_{rs} + 1) \times (\kappa_{rs} + 1)$ matrix. The recurrence takes constant work per entry, and evaluating the normalization also takes $O((\kappa_{rs} + 1)^2)$ operations. The weights v_{rs}^t and μ_k are obtained from the factorial and binomial tables above. Preparing all Hahn transform matrices therefore requires

$$\sum_{r,s=0}^n O((\kappa_{rs} + 1)^2) = O(n^4) \quad (5.23)$$

arithmetic operations. With these matrices prepared, multiplication by U or U^T uses $(\kappa_{rs} + 1)^2$ multiplications and $\kappa_{rs}(\kappa_{rs} + 1)$ additions for each pair r, s . The coordinate rescalings require $O(n^3)$ further operations in total. Thus each conversion takes $O(n^4)$ operations in either direction, with or without preparation. Retaining all transform matrices requires $O(n^4)$ storage. The $O(n^{1+\tau})$ bound applies to the shared-factor algorithm, whereas preparing the Hahn transform matrices already requires $O(n^4)$ operations.

5.3 The composed Schur-Pauli transformation

Combining the preceding transformations gives a conversion between Pauli orbit coefficients and Schur blocks. Both use the same matrix-unit orbit sums, but we do need to do some relabeling. By [Equations \(4.18\)](#) and [\(5.3\)](#),

$$M_{t,s-t}^{(r+s-2t)} = a_{rs}^t. \quad (5.24)$$

Corollary 5.6 (Schur–Pauli conversion). *For $A \in \text{End}(\mathcal{H}_n)^{S_n}$, its Pauli orbit coefficients and representative Schur blocks can be converted into one another in $O(n^4)$ arithmetic operations, including preprocessing. Using a fast matrix multiplication algorithm with exponent τ , the conversion requires $O(n^{1+\tau})$ operations.*

Proof. Starting from the Pauli orbit coefficients, first apply Equation (4.23) to obtain the matrix-unit orbit coefficients, then Theorem 5.3 to compute the Schur blocks. In the reverse direction, first use the inverse conversion in Theorem 5.3, then apply Equation (4.13) to obtain the Pauli orbit coefficients.

There are $O(n^3)$ intermediate coefficients, so relabeling costs $O(n^3)$ operations. By Theorems 4.3 and 5.4, each stage costs $O(n^4)$ operations, or $O(n^{1+\tau})$ with fast matrix multiplication, including preprocessing. Adding the two costs gives the stated bound. $\square$

This completes the bidirectional conversion between all three compressed descriptions and the main result of this thesis.

Chapter 6

Applications

The Schur blocks reduce spectral computations and invariant dynamics to matrices of dimension at most $n + 1$. The transformations in [Chapters 4 and 5](#) provide these blocks from Pauli or computational orbit coordinates. We benchmark these immediate applications (eigensystems and dynamics) using randomized inputs and consider the fully connected Ising model as a concrete physical example. Throughout this chapter, we work in units where $\hbar = 1$.

6.1 Spectral computations

6.1.1 Eigenvalues and eigenstates

Let $H \in \text{End}(\mathcal{H}_n)^{S_n}$ be self-adjoint, with Schur blocks H_k of dimensions d_k and multiplicities μ_k as in [Corollary 2.26](#). Choose an orthonormal eigenbasis $\{|v_{k,j}\rangle\}_{j=1}^{d_k}$ of each block matrix H_k , satisfying $H_k|v_{k,j}\rangle = E_{k,j}|v_{k,j}\rangle$. Under the Schur basis identification of [Definition 2.27](#), an orthonormal eigenbasis of H is then given by

$$|\psi_{k,j,a}\rangle = |v_{k,j}\rangle \otimes |a\rangle_k = \sum_{r=k}^{n-k} \langle k, r | v_{k,j} \rangle |k, r, a\rangle, \quad 1 \leq a \leq \mu_k. \quad (6.1)$$

Each block eigenvector gives μ_k orthogonal eigenstates of the full Hamiltonian with eigenvalue $E_{k,j}$. Details are given in [Section A.6](#).

The conversion to the Schur blocks costs $O(n^4)$ arithmetic operations with our implementation. Using conventional eigensolvers for the blocks, such as the routines in LAPACK [[And+99](#), Sections 2.4.4 and 3.4.3], gives

$$O(n^4) + \sum_{k=0}^{\lfloor n/2 \rfloor} O(d_k^3) = O(n^4) \quad (6.2)$$

for the numerical eigenvalues and eigenvectors in Schur coordinates. Fast matrix multiplication improves the arithmetic bounds for both the conversion (see [Section 5.2.2](#)) and the diagonalization.

A precise theoretical bound follows from [[Sob25](#), Corollary 2.4]. After rescaling so that $\|H\|_\infty \leq 1$, fix $0 < \varepsilon < 1/2$ and a matrix multiplication exponent $2 < \tau < 3$. Their algorithm computes matrices $\tilde{V}_k$ and real diagonal matrices $\tilde{\Lambda}_k$ satisfying

$$\begin{aligned} \|H_k - \tilde{V}_k \tilde{\Lambda}_k \tilde{V}_k^\dagger\|_\infty &\leq \varepsilon, \\ \|\tilde{V}_k^\dagger \tilde{V}_k - I_{d_k}\|_\infty &\leq \varepsilon/d_k^2. \end{aligned} \quad (6.3)$$

Summing the algorithm's cost over the $O(n)$ blocks of dimension at most $n+1$, and including the fast conversion from [Corollary 5.6](#), gives

$$O\left(n^{1+\tau} \log(n+2) + n^3 \operatorname{polylog}\left(\frac{n+2}{\varepsilon}\right)\right) \quad (6.4)$$

arithmetic operations and comparisons. Here the conversion is evaluated in exact arithmetic. The block errors do not accumulate across the direct sum, so the resulting approximation of H also has spectral norm error at most ε . In a floating-point implementation, conversion error must be accounted for separately.

Remark 6.1 (Quantum preparation of eigenstates). Given a classical description of the normalized block eigenvector $|v_{k,j}\rangle$ to sufficient accuracy, prepare the Schur register state $|k\rangle|v_{k,j}\rangle|a\rangle_k$ with a fixed valid multiplicity label a . Using compatible Schur-basis conventions, the inverse quantum Schur transform [\[BCH05\]](#) prepares the corresponding n -qubit eigenstate $|\psi_{k,j,a}\rangle$ to Euclidean-norm error at most ε . The block vector has at most $n+1$ coefficients, and both preparation steps require resources polynomial in n and $\log(1/\varepsilon)$. For more details on preparation we refer to [\[Ans+23\]](#).

6.1.2 Computing only selected eigenvalues

For the ground-state energy,

$$E_0 = \min_k \lambda_{\min}(H_k), \quad (6.5)$$

it is enough to compute a lowest eigenvalue of each block. For concrete Hamiltonians with few Pauli-orbit terms, constructing their blocks directly can also be cheaper than preparing our general conversion. Reusing the conversion tables across several Hamiltonians, however, can make the general method the more efficient choice.

If a fixed sector index k is known to contain the desired eigenstate, its block can be constructed from matrix-unit orbit coefficients in $O((k+1)n^3)$ operations. Starting from general Pauli orbit coefficients still requires the preceding $O(n^4)$ conversion. Details are given in [Section A.6](#).

6.2 Invariant dynamics

Let $M \in \operatorname{End}(\mathcal{H}_n)^{S_n}$ be Hermitian, let ρ be the initial state, and let $U(t)$ be a permutation-invariant time evolution operator. For a continuously differentiable unitary evolution with $U(0) = I$, permutation invariance of $U(t)$ at every time is equivalent to permutation invariance of its Hamiltonian generator $H(t) = i\dot{U}(t)U(t)^\dagger$. We want to compute

$$f(t) = \operatorname{Tr}\left[MU(t)\rho U(t)^\dagger\right]. \quad (6.6)$$

The state ρ need not be permutation invariant. We can, however, replace it by its group average without changing $f(t)$. Using [Definition 2.16](#), define the symmetrized state

$$\bar{\rho} := \mathcal{A}_{S_n}(\rho) = \frac{1}{n!} \sum_{\pi \in S_n} U_\pi \rho U_\pi^\dagger. \quad (6.7)$$

Since both $U(t)$ and M commute with every U_π , the evolved observable $M(t) := U(t)^\dagger M U(t)$ is also permutation invariant. By cyclicity of the trace,

$$\begin{aligned} \operatorname{Tr}[\bar{\rho}M(t)] &= \frac{1}{n!} \sum_{\pi \in S_n} \operatorname{Tr}[\rho U_\pi^\dagger M(t) U_\pi] \\ &= \operatorname{Tr}[\rho M(t)] = f(t). \end{aligned} \quad (6.8)$$

By [Corollary 2.26](#), the invariant operators $\bar{\rho}$ and $U(t)$ have block forms

$$\bar{\rho} \cong \bigoplus_k \bar{\rho}_k \otimes I_{\mu_k}, \quad U(t) \cong \bigoplus_k U_k(t) \otimes I_{\mu_k},$$

where each $U_k(t)$ is unitary. Taking the trace in the Schur basis therefore gives

$$f(t) = \sum_k \mu_k \operatorname{Tr} \left[M_k U_k(t) \bar{\rho}_k U_k(t)^\dagger \right]. \quad (6.9)$$

For simplicity, set

$$\tau_k := \mu_k \bar{\rho}_k, \quad \tau_k \geq 0, \quad \sum_k \operatorname{Tr}(\tau_k) = 1. \quad (6.10)$$

Each τ_k acts on $\mathcal{W}_k$ and has dimension $d_k = n - 2k + 1$. With this notation,

$$f(t) = \sum_k \operatorname{Tr} \left[\tau_k U_k(t)^\dagger M_k U_k(t) \right]. \quad (6.11)$$

This equation matches the work by Anschuetz et al. [[Ans+23](#), Lemma 3]. If ρ is already permutation invariant, then $\bar{\rho} = \rho$ and $\tau_k = \mu_k \rho_k$. In either case, the reduced matrices contain only $\sum_k d_k^2 = \binom{n+3}{3}$ entries.

6.2.1 Classically supplied state data

Suppose first that the reduced matrices τ_k are given, or that the symmetrized state $\bar{\rho}$ is supplied in Pauli or computational orbit coordinates. We convert M to its Schur blocks using the transformations of the preceding chapters. In the second case, we also convert $\bar{\rho}$ and set $\tau_k = \mu_k \bar{\rho}_k$. Reading an arbitrary dense matrix ρ already requires exponentially many operations.

If $U(t)$ is supplied in either orbit coordinates or directly by its Schur blocks, evaluating [Equation \(6.11\)](#) by block multiplication costs $\sum_k O(d_k^3) = O(n^4)$ operations per requested time. Fast matrix multiplication (in conversion and calculation) reduces this cost to $O(n^{1+\tau})$. Computing $U(t)$ from a time-dependent Hamiltonian requires additional operations:

For a time-independent Hermitian $H \in \operatorname{End}(\mathcal{H}_n)^{S_n}$, we set $U(t) = e^{-itH}$. After converting H to its Schur blocks, we can reuse the eigenvectors from [Section 6.1](#) at every requested time. Let V_k be the change-of-basis matrix into the eigenbasis. We write

$$H_k = V_k \operatorname{diag}(E_{k,1}, \dots, E_{k,d_k}) V_k^\dagger, \\ M'_k := V_k^\dagger M_k V_k, \quad \tau'_k := V_k^\dagger \tau_k V_k. \quad (6.12)$$

In this basis, time evolution multiplies the (j, ℓ) entry of M'_k by $e^{it(E_{k,j} - E_{k,\ell})}$. Inserting this into [Equation \(6.11\)](#) gives

$$f(t) = \sum_{k=0}^{\lfloor n/2 \rfloor} \sum_{j,\ell=1}^{d_k} (M'_k)_{j,\ell} (\tau'_k)_{\ell,j} e^{it(E_{k,j} - E_{k,\ell})}. \quad (6.13)$$

The products $(M'_k)_{j,\ell} (\tau'_k)_{\ell,j}$ and energy differences $E_{k,j} - E_{k,\ell}$ do not depend on t and can be stored once. For each requested time, we then evaluate the phase factors and sum the $O(n^3)$ terms in [Equation \(6.13\)](#).

Converting all necessary operators to Schur blocks costs $O(n^4)$ operations. With the conventional eigensolvers discussed in [Section 6.1](#), diagonalizing the blocks H_k and expressing

M_k and τ_k in the eigenbasis of H_k also costs $\sum_k O(d_k^3) = O(n^4)$. Thus evaluating $f(t)$ at L prescribed times requires

$$O(n^4 + Ln^3) \quad (6.14)$$

operations in total, with the first term accounting for preparation and the second for the L evaluations. Here we also assume that evaluating phase factors $e^{it(E_{k,j} - E_{k,\ell})}$ costs $O(1)$. Using fast conversion, eigensolvers and matrix multiplication for the basis changes reduces the preparation cost to the bound in Equation (6.4), while the $O(Ln^3)$ evaluation cost remains unchanged.

For Hermitian operators $\tilde{H}$ and H , Duhamel's formula gives $\|e^{-it\tilde{H}} - e^{-itH}\|_\infty \leq |t|\|\tilde{H} - H\|_\infty$. With an exact normalized state and observable, this bounds the Hamiltonian-induced expectation error by $2|t|\|M\|_\infty\|\tilde{H} - H\|_\infty$. The checks in Section C.3 measure the total, combined error, including errors in the observable and state, in the tested regime.

6.2.2 Access through quantum copies

We can also estimate $f(t)$ from repeated preparations of an unknown state ρ on a quantum device. The quantum Schur transform implements the change from the computational basis to registers recording the sector k , the basis label in $\mathcal{W}_k$, and the multiplicity label in $\mathcal{S}_k$ [BCH05]. Measuring the k -register gives outcome k with the probability $p_k = \text{Tr}(\tau_k)$, afterwards we may discard the multiplicity register and work with the reduced state τ_k/p_k on $\mathcal{W}_k$.

We therefore have reduced the state to $q = \lceil \log_2(n+1) \rceil$ qubits. On each of these qubits choose an independent, uniformly random Clifford C_a [HKP20], apply it to the state, and record the computational-basis outcome b_a . The following reconstruction map assigns to the recorded Clifford operators and outcomes the matrix

$$\hat{\sigma} = \bigotimes_{a=1}^q \left(3C_a^\dagger |b_a\rangle\langle b_a| C_a - I_2 \right). \quad (6.15)$$

This matrix is an unbiased estimator of the padded conditional state: averaging over the random Clifford choices and their outcomes within sector k gives

$$\mathbb{E}[\hat{\sigma} \mid k] = \frac{\tau_k}{p_k} \oplus 0.$$

For the recorded sector k , the estimator at time t_j is $\text{Tr}[\hat{\sigma}(M_k(t_j) \oplus 0)]$, where $M_k(t_j) = U_k(t_j)^\dagger M_k U_k(t_j)$. Its expectation over all records, including the sector outcomes, is $f(t_j)$. The reconstruction is classical post-processing of the measurement records and uses the full padded register, including measurement outcomes outside the original d_k basis states.

For prescribed times $t_1, \dots, t_L$, we apply the result of Chang, Larocca and Cerezo [CLC26, Lemma 1 and Appendix D, Theorem 4] to the invariant observables $M(t_1), \dots, M(t_L)$. We split independent records into $O(\log(2L/\delta))$ groups, average each estimator within each group, and take the median of those means. Since unitary conjugation preserves the spectral norm, $\|M(t_j)\|_\infty = \|M\|_\infty$, their result gives the sufficient number of copies

$$N = O\left(\frac{n^2\|M\|_\infty^2}{\varepsilon^2} \log \frac{2L}{\delta}\right) \quad (6.16)$$

to estimate all L values to additive error $\varepsilon > 0$, with probability at least $1 - \delta$.

The same records are used for all requested times. The local-Clifford variance bound is $O(4^q\|M\|_\infty^2) = O(n^2\|M\|_\infty^2)$, so padding changes only a constant in Equation (6.16).

The quantum Schur transform is an additional operation on each copy, with gate count $n \text{ poly}(\log n, \log(1/\eta))$ [BCH05, Sections IV and V.C]. Here we require unitary implementations Q and $\tilde{Q}$ on the same registers, including any extra qubits prepared in a fixed initial state, with $\|\tilde{Q} - Q\|_\infty \leq \eta$. The resulting state trace-norm error is at most 2η , and hence the observable bias is at most $2\eta\|M\|_\infty$. For $M \neq 0$, choosing $\eta \leq \varepsilon/(4\|M\|_\infty)$ and statistical error $\varepsilon/2$ gives total error at most ε with the same asymptotic copy bound.

Two additional approaches we investigated are summarized in the remark below. They are, in general, asymptotically less efficient than the above method, but may be of interest in other contexts.

Remark 6.2. One approach is full tomography of the reduced state $\tilde{\rho} := \bigoplus_k \tau_k$, of dimension $\sum_k d_k = \Theta(n^2)$. Applying general tomography gives a sufficient bound of $\tilde{O}(n^4\|M\|_\infty^2/\varepsilon^2)$ copies for expectation error ε at fixed success probability [Haa+17], with a worse dependence on n than Equation (6.16).

Another approach is Pauli sampling: Fix t and write $M(t) = \sum_{P \in \mathcal{P}_n} c_P P$, with coefficient vector c and m nonzero terms. We employ the Pauli strings with a probability distribution given by the normalized absolute values of the coefficients, $p_P = |c_P|/\|c\|_1$ or uniformly over the nonzero terms, $p_P = 1/m$ [Had+22].

Sampling a string P accordingly and measuring its outcome $s \in \{\pm 1\}$ gives the unbiased estimator $c_P s/p_P$. Hoeffding's inequality gives the respective sufficient copy bounds

$$N_{\text{unif}} = O\left(\frac{m^2\|c\|_\infty^2}{\varepsilon^2} \log \frac{2}{\delta}\right), \quad N_{\ell_1} = O\left(\frac{\|c\|_1^2}{\varepsilon^2} \log \frac{2}{\delta}\right). \quad (6.17)$$

These guarantees depend on the Pauli coefficients. Their ℓ_1 norm can grow exponentially with n even when $\|M\|_\infty = 1$, and permutation invariance alone also does not bound the number m of nonzero Pauli strings polynomially in n .

6.3 Numerical applications

In the following section, we benchmark the preceding applications on random inputs and then consider the fully connected Ising model as a concrete physical example. We compare our conversion with the optimized combinatorial implementation of Anschuetz et al., full-space diagonalization for small systems, and direct PIQS block construction for the Ising model. The benchmarks use the Hahn implementation in Remark 5.5.

The benchmark details and additional accuracy checks are given in Section C.3 and the GitHub repository [Sau26].

6.3.1 Random spectral computations

For random Hermitian Hamiltonians, we construct all distinct Schur blocks and compute their eigensystems (eigenvalues and eigenvectors), to test whether our conversion can reduce the effort of diagonalizing a permutation-invariant Hamiltonian. For $n \leq 10$, we also construct and diagonalize the full Hamiltonian in the computational basis¹. This provides a baseline for the benefit of symmetry reduction, while the comparison with Anschuetz isolates the benefit of our faster Schur conversion. We restrict the PIQS comparison to the Ising example, where the inputs have simple collective-spin expressions. For the general random Hamiltonians considered here, using PIQS would require an additional conversion from Pauli-orbit coefficients. In Figure 6.1, the runtime advantage of our conversion grows with n over the tested range, showing that faster block construction also benefits the complete

¹Larger n are not included because of time constraints.

eigensystem calculation. Full-space diagonalization is faster for the smallest systems, but its runtime grows more rapidly over the tested range. In the example spectrum, the energy range narrows as k increases.

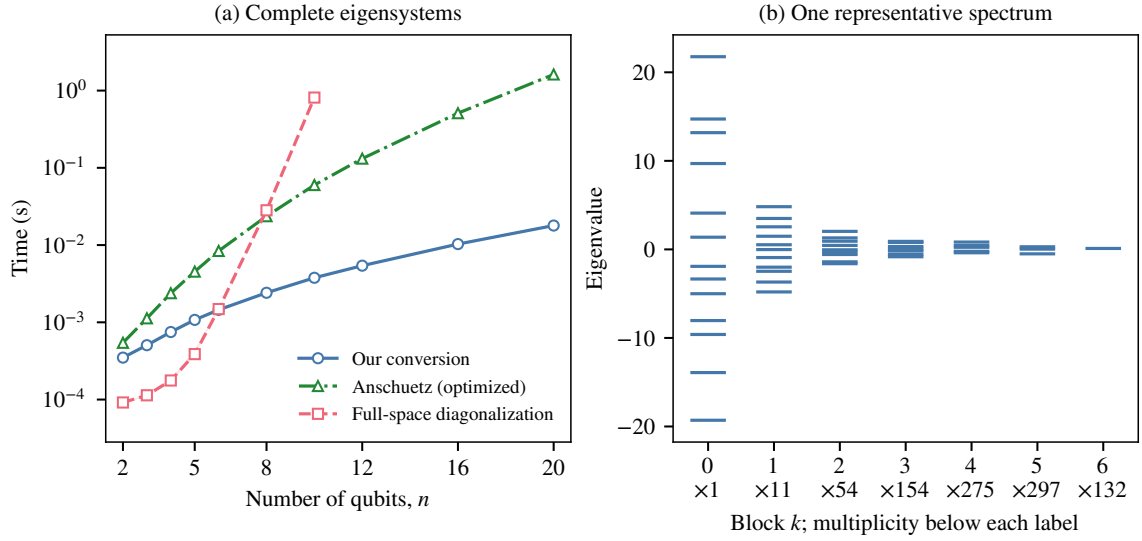


Figure 6.1: Left: time to construct the Hamiltonian and compute its complete eigensystem, including preprocessing. The full-space baseline constructs and diagonalizes the $2^n \times 2^n$ matrix for $n \leq 10$. Markers and shading show medians and interquartile ranges. Right: the block eigenvalues of one random input at $n = 12$.

6.3.2 Random invariant dynamics

For dynamics, we independently sample the Hamiltonian, observable and a positive invariant initial state. We construct the necessary Schur blocks and evaluate Equation (6.13) at the required times. Figure 6.2 compares the total runtime as the system size and the number of requested times vary. The two sweeps separate growth with system size from the effect of reusing the same blocks and eigensystems for more time points. For a fixed number of times, the advantage of our conversion grows with system size over the tested range. As more times are requested, evaluating the phase sum accounts for a larger fraction of the runtime in both methods, so the relative advantage of faster conversion decreases.

6.3.3 The fully connected Ising model

As a concrete physical example, consider $n \geq 2$ spins with an Ising interaction between every pair and a transverse, uniform magnetic field,

$$H = -\frac{1}{n} \left(g \sum_{i < j} Z_i Z_j + h \sum_i X_i \right). \quad (6.18)$$

The factor $1/n$ scales both the interaction and the field term. Thus, at fixed g and h , the field acting on each spin decreases with n . This Hamiltonian has at most two nonzero Pauli-orbit coefficients: $-g/n$ for the orbit with two Z factors, and $-h/n$ for the orbit with one X factor. Since this Hamiltonian has fixed locality, its nonzero Schur-block entries can be constructed directly in $O(n^2)$ operations by Theorem 3.7. We use it here to illustrate

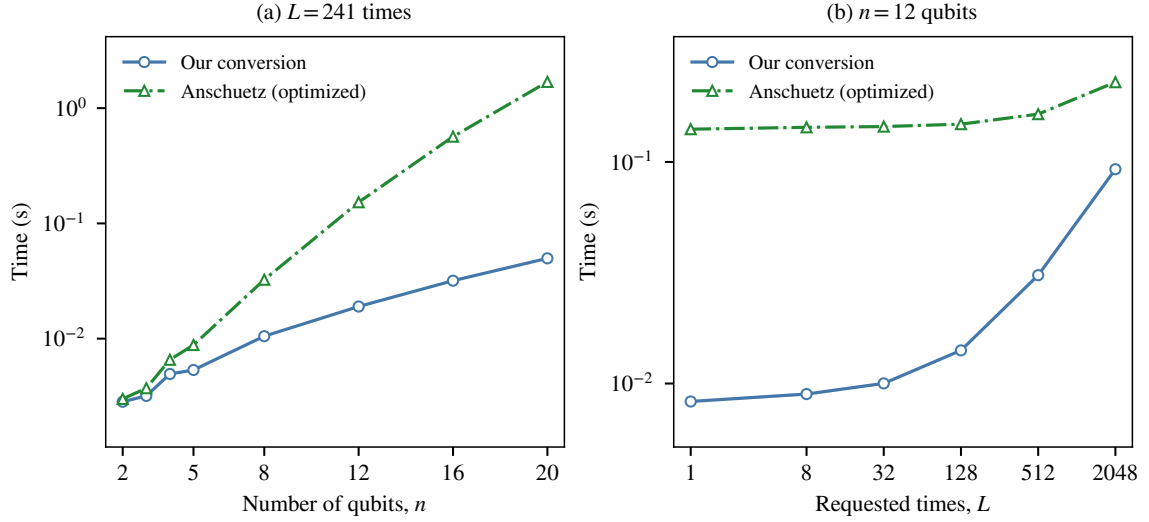


Figure 6.2: Complete runtime for random invariant dynamics, including preparation. Left: varying the system size for a fixed selection of evaluation times. Right: varying the number of requested times at a fixed number of qubits. Markers and shading show medians and interquartile ranges.

the spectral and dynamics calculations obtained from our general conversion. Diagonalizing these blocks gives the spectrum in [Figure 6.3](#).

For the dynamics, take an initial state and measure the transverse magnetization,

$$\rho(0) = \left(\frac{I + pX}{2} \right)^{\otimes n}, \quad |p| \leq 1, \quad M = \frac{1}{n} \sum_i X_i. \quad (6.19)$$

The coefficient of $\rho(0)$ on the orbit with a factors X and $n - a$ factors I is $2^{-n} p^a$ and all other coefficients vanish. The observable has the coefficient $1/n$ on the one- X orbit. These values provide the inputs for [Equation \(6.13\)](#).

In [Figure 6.3](#), the lowest energy occurs in the $k = 0$ block. The magnetization minimum shifts to later times as n increases. The $n = 8$ curve also shows a pronounced revival within the displayed interval.

For this model, we also compare our conversion with direct block construction using the collective-spin operators supplied by PIQS [\[Sha+18\]](#). This direct construction relies on simple collective-spin expressions for the inputs. Both methods use the same block diagonalization and spectral evolution routine. [Figure 6.4](#) shows that direct construction becomes faster for the larger tested systems, reducing both preparation and total runtime. This shows that for concrete Hamiltonians, there may often exist faster methods to calculate the desired quantities than our general conversion method.

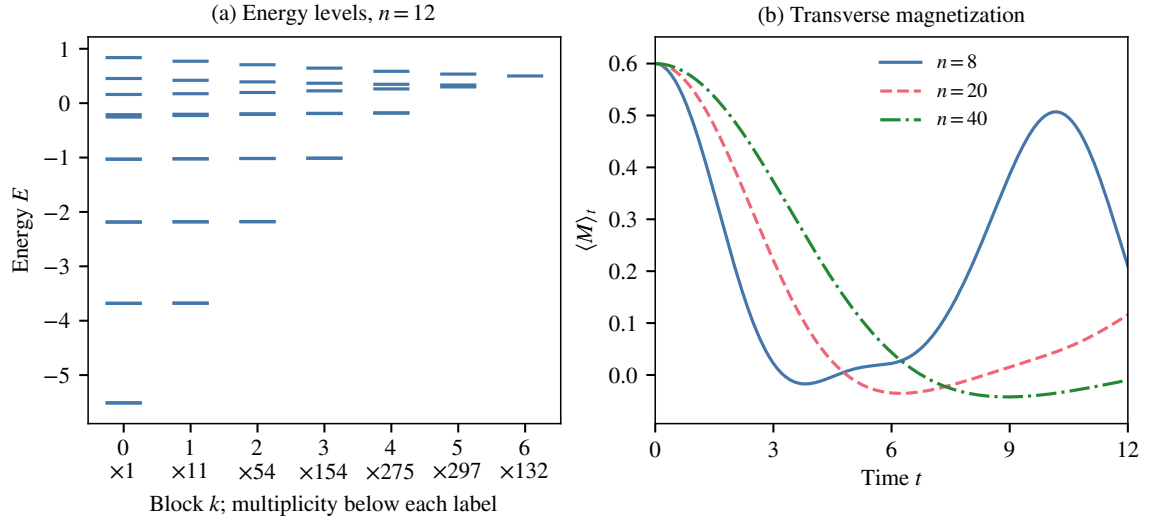


Figure 6.3: The fully connected Ising model with $g = 1$ and $h = 1/2$. Left: all block eigenvalues at $n = 12$. Right: transverse magnetization from Equation (6.19) with $p = 0.6$, for $n = 8, 20, 40$.

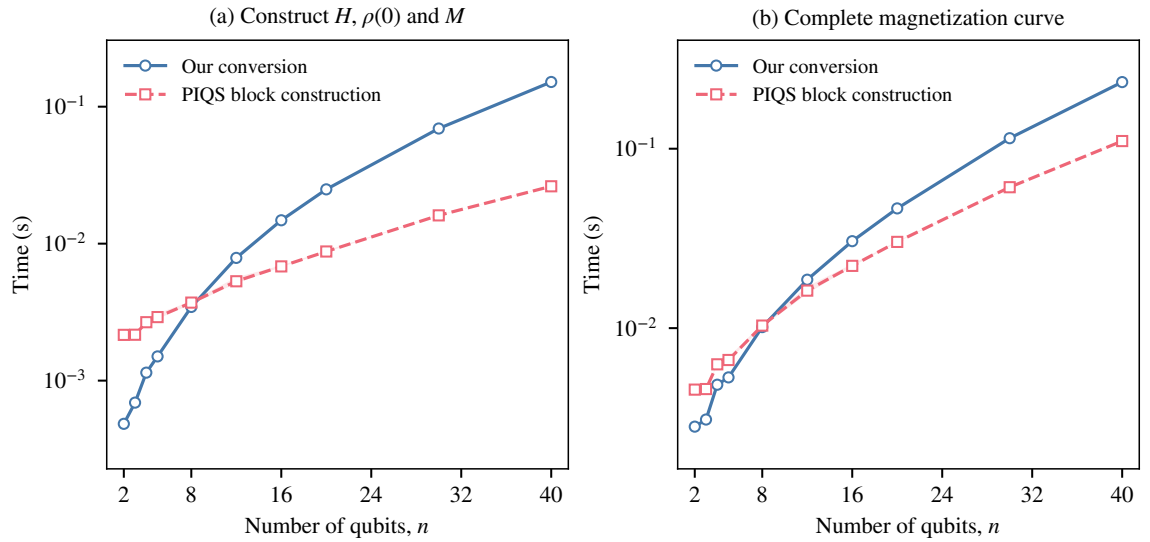


Figure 6.4: Ising runtime comparison at $g = 1$, $h = 1/2$ and $p = 0.6$. Left: construction of the Hamiltonian, state and observable blocks. Right: complete runtime including construction, block diagonalization and 241 magnetization values at equally spaced times in $[0, 12]$. Markers and shading show medians and interquartile ranges over seven repetitions.

Chapter 7

Conclusion and outlook

7.1 Conclusion

We have constructed bidirectional conversions between matrix-unit orbit coefficients, Pauli orbit coefficients and Schur blocks of permutation-invariant operators. The Schur representation is particularly suited to classical matrix computations with permutation-invariant operators, since matrix products, functions of operators and positivity tests can all be performed blockwise. Spectral computations and simulating invariant dynamics are two immediate applications of this structure. Each conversion uses $O(n^4)$ arithmetic operations including preprocessing, or $O(n^{1+\tau})$ with fast matrix multiplication. The constructions build on prior work, mainly in coding theory [Gij09] and combinatorics [FK07]. The Python implementation and its numerical checks are available at [Sau26]. Our numerical comparisons illustrate improvements in the efficiency of spectral computations and invariant dynamics of symmetric quantum systems, when the required compressed input data is available, while additional assumptions, such as fixed locality, may allow further improvements [CLC26].

7.2 Outlook

The main limitations of our work are the physically limiting assumption of exact permutation invariance and the computational assumption of a compressed input form. Other forms of input should be analyzed carefully. Practical advantages of our algorithms also depend on locality, the possibility of reusing preprocessing and the concrete physical model analyzed. Improvements on these limitations are an interesting direction for future work. In the following sections, we give an overview of some concrete improvements and generalizations of our work, as well as some open questions.

7.2.1 Fast polynomial transforms

The separated Krawtchouk transforms in Theorem 4.2 and the Hahn formulation in Remark 5.5 suggest further improvements through fast polynomial algorithms. Bostan, Salvy and Schost [BSS10, Theorem 1] give conversions between a polynomial basis satisfying a nondegenerate three-term recurrence (such as Krawtchouk and Hahn polynomials [Nat]) and the monomial basis $1, x, \dots, x^{m-1}$ in $O(M(m) \log m)$ arithmetic operations, where $M(m)$ denotes the cost of multiplying polynomials of degree less than m . With standard FFT multiplication, this gives $O(m \log^2 m)$.

Applying this result to our conversions would require combining polynomial basis changes with the relevant evaluation maps and weights. If the resulting Krawtchouk and Hahn

transforms and their inverses can be computed in $O(M(m) \log m)$ operations, including preparation, then the $O(n^2)$ transforms of length at most $n + 1$ give a total of

$$O(n^3 \log^2 n) \quad (7.1)$$

operations using standard FFT multiplication. This would bring the conversion costs in [Theorems 4.3](#) and [5.4](#) within logarithmic factors of the output-size lower bound in [Remark 4.4](#).

A recent preprint by OpenAI [[Ope26a](#), Corollaries 1.2–1.3] reports algorithms for the exact discrete Fourier transform and polynomial multiplication using $O(m(\log m)^{1-\varepsilon})$ operations, with $\varepsilon = 10^{-13}$. The model uses exact complex arithmetic with unrestricted coefficients and access to specified roots of unity, and includes scalar preparation. With the required roots available, using this bound for $M(m)$ would improve the conditional total above to $O(n^3(\log n)^{2-\varepsilon})$.

Numerical stability and precision requirements may limit the practical usefulness of these approaches. We were not able to implement such an approach with the desired accuracy or speedup within the time frame of this thesis.

Proving lower bounds beyond the output-size bound remains an interesting question. The Fourier transform illustrates why the computational model matters: $\Omega(m \log m)$ lower bounds are known for the unnormalized transform with bounded coefficients [[Mor73](#)] and in Ailon’s model with uniformly bounded intermediate condition numbers [[Ail14](#)]. Any implication for Fourier-transform optimality would require a reduction relating the two problems within the same model.

7.2.2 Generalizing to qudits

For qudits of a local dimension d , the four qubit Pauli matrices must be replaced by a basis of d^2 local $d \times d$ operators. The same argument from [[KLC24](#)] then gives:

$$\dim \text{End}((\mathbb{C}^d)^{\otimes n})^{S_n} = \binom{n + d^2 - 1}{d^2 - 1}, \quad (7.2)$$

which is polynomial in n for fixed d (see also [[Ans+23](#), Appendix G]). Polynomial-time conversion between matrix-unit orbit coefficients and Schur blocks is already available for fixed local dimension d , through Gijswijt’s construction [[Gij09](#), Theorem 6] and its presentation by Bergh and Parentin [[BP26c](#), Section III.2 and Appendix A.2]. A further question is whether the separated transforms and shared-factor computations used here admit analogous constructions for qudits, and what arithmetic complexity these would achieve.

7.2.3 Generalizing to other groups

For other symmetry groups, efficient conversion depends on the dimensions and multiplicities of their irreducible representations. The examples in [Section A.7](#) show that cyclic symmetry alone still leaves exponentially large blocks. Further applications would therefore need additional structure or a different choice of symmetry.

7.2.4 Sparse Fourier reconstruction

Another direction is to investigate whether sparse Fourier reconstruction can improve Spencer et al.’s Pauli decomposition algorithm under the same sparse query access model. See [Remark 3.5](#) for a small discussion.

7.2.5 Quantum channel fidelity and semidefinite programming

Finally, we note that, as shown by Bergh and Parentin [BP26c, Sections II–IV], the Schur basis is particularly useful for semidefinite programs with permutation symmetry. Their *symmetric seesaw* method optimizes encoding and decoding maps within permutation-invariant codes to obtain lower bounds on the channel fidelity over n parallel channel uses. Potentially our matrix-unit to Pauli orbit conversion could be used in applications with Pauli noise, but further investigation remains necessary.

Bibliography

- [ABK23] Eric R. Anschuetz, Andreas Bauer, and Bobak Kiani. *An efficient classical algorithm for simulating symmetric quantum systems*. 2023. URL: https://github.com/bkiani/symmetric_hamiltonians.
- [Ail14] Nir Ailon. *An $n \log n$ Lower Bound for Fourier Transform Computation in the Well Conditioned Model*. 2014. arXiv: [1403.1307](https://arxiv.org/abs/1403.1307) [cs.CC]. URL: <https://arxiv.org/abs/1403.1307>.
- [AL18] Tameem Albash and Daniel A. Lidar. “Adiabatic quantum computation”. In: *Rev. Mod. Phys.* 90 (1 Jan. 2018), p. 015002. DOI: [10.1103/RevModPhys.90.015002](https://link.aps.org/doi/10.1103/RevModPhys.90.015002). URL: <https://link.aps.org/doi/10.1103/RevModPhys.90.015002>.
- [And+99] E. Anderson et al. *LAPACK Users’ Guide*. 3rd ed. Philadelphia, PA: Society for Industrial and Applied Mathematics, 1999. ISBN: 0-89871-447-8. URL: <https://www.netlib.org/lapack/lug/>.
- [Ans+23] Eric R. Anschuetz, Andreas Bauer, Bobak T. Kiani, and Seth Lloyd. “Efficient classical algorithms for simulating symmetric quantum systems”. In: *Quantum* 7 (Nov. 2023), p. 1189. ISSN: 2521-327X. DOI: [10.22331/q-2023-11-28-1189](https://dx.doi.org/10.22331/q-2023-11-28-1189). URL: <http://dx.doi.org/10.22331/q-2023-11-28-1189>.
- [Arn16] Andrew Arnold. “Sparse Polynomial Interpolation and Testing”. PhD thesis. University of Waterloo, 2016. URL: <https://hdl.handle.net/10012/10307>.
- [Arv76] William B. Arveson. *An Invitation To C^* -Algebras*. Springer, 1976. DOI: [10.1007/978-1-4612-6371-5](https://doi.org/10.1007/978-1-4612-6371-5). URL: <https://doi.org/10.1007/978-1-4612-6371-5>.
- [BB17] Jacob Biamonte and Ville Bergholm. *Tensor Networks in a Nutshell*. 2017. arXiv: [1708.00006](https://arxiv.org/abs/1708.00006) [quant-ph]. URL: <https://arxiv.org/abs/1708.00006>.
- [BCH05] Dave Bacon, Isaac L. Chuang, and Aram W. Harrow. *The Quantum Schur Transform: I. Efficient Qudit Circuits*. 2005. arXiv: [quant-ph/0601001](https://arxiv.org/abs/quant-ph/0601001) [quant-ph]. URL: <https://arxiv.org/abs/quant-ph/0601001>.
- [Bou98] N. Bourbaki. *Algebra I: Chapters 1-3*. Actualités scientifiques et industrielles. Springer, 1998. ISBN: 9783540642435. URL: <https://books.google.de/books?id=STS9aZ6F204C>.
- [BP26a] Bjarne Bergh and Marco Parentin. *permqit: A Toolbox for Permutation Invariant Optimization Problems in Quantum Information Theory*. GitHub repository. Version 0.1.0, commit 22af3cd. 2026. URL: <https://github.com/bbbergh/permqit>.

- [BP26b] Bjarne Bergh and Marco Parentin. *permqit: A Toolbox for Permutation Invariant Optimization Problems in Quantum Information Theory*. GitHub repository. Version 0.1.0, commit 22af3cd. 2026. URL: <https://github.com/bbbergh/permqit/tree/22af3cd245bd0e950df49f6ce16ccd422b6eb2c5> (visited on 09/19/2026).
- [BP26c] Bjarne Bergh and Marco Parentin. *Permutation Invariant Optimization Problems in Quantum Information Theory: A Framework for Channel Fidelity and Beyond*. 2026. arXiv: 2604.27040 [quant-ph]. URL: <https://arxiv.org/abs/2604.27040>.
- [Bre70] R. P. Brent. *Algorithms for Matrix Multiplication*. Tech. rep. STAN-CS-70-157. Department of Computer Science, Stanford University, Mar. 1970. URL: <https://maths-people.anu.edu.au/~brent/pd/rpb002i.pdf>.
- [BSS10] Alin Bostan, Bruno Salvy, and Éric Schost. “Fast conversion algorithms for orthogonal polynomials”. In: *Linear Algebra and its Applications* 432.1 (2010), pp. 249–258. DOI: 10.1016/j.laa.2009.08.002. arXiv: 0804.2373 [cs.SC]. URL: <https://doi.org/10.1016/j.laa.2009.08.002>.
- [CLC26] Su Yeon Chang, Martin Larocca, and M. Cerezo. *Practical framework for simulating permutation-equivariant quantum circuits*. 2026. arXiv: 2603.13072v1 [quant-ph]. URL: <https://arxiv.org/abs/2603.13072v1>.
- [CMA21] Gabriele Carcassi, Lorenzo Maccone, and Christine A. Aidala. “Four Postulates of Quantum Mechanics Are Three”. In: *Phys. Rev. Lett.* 126 (11 Mar. 2021), p. 110402. DOI: 10.1103/PhysRevLett.126.110402. URL: <https://link.aps.org/doi/10.1103/PhysRevLett.126.110402>.
- [Col11] Rodney Coleman. *On Krawtchouk polynomials*. 2011. arXiv: 1101.1798 [math.CA]. URL: <https://arxiv.org/abs/1101.1798>.
- [Dal+25] Alexander M. Dalzell et al. *Quantum Algorithms: A Survey of Applications and End-to-end Complexities*. Cambridge University Press, Apr. 2025. ISBN: 9781009639668. DOI: 10.1017/9781009639651. URL: <http://dx.doi.org/10.1017/9781009639651>.
- [DF03] D.S. Dummit and R.M. Foote. *Abstract Algebra*. Wiley, 2003. ISBN: 9780471433347. URL: <https://books.google.de/books?id=KJDBQgAACAAJ>.
- [FH13] W. Fulton and J. Harris. *Representation Theory: A First Course*. Graduate Texts in Mathematics. Springer New York, 2013. ISBN: 9781461209799. URL: <https://books.google.de/books?id=6TwmBQAAQBAJ>.
- [FK07] Philip Feinsilver and Jerzy Kocik. *Krawtchouk polynomials and Krawtchouk matrices*. 2007. arXiv: quant-ph/0702073 [quant-ph]. URL: <https://arxiv.org/abs/quant-ph/0702073>.
- [Geo+24] Timothy N. Georges, Bjorn K. Berntson, Christoph Sünderhauf, and Aleksei V. Ivanov. *Pauli decomposition via the fast Walsh-Hadamard transform*. 2024. URL: https://github.com/riverlane/pauli_lcu.
- [Geo+25a] Timothy N Georges, Bjorn K Berntson, Christoph Sünderhauf, and Aleksei V Ivanov. “Pauli decomposition via the fast Walsh-Hadamard transform”. In: *New Journal of Physics* 27.3 (Feb. 2025), p. 033004. ISSN: 1367-2630. DOI: 10.1088/1367-2630/adb44d. URL: <http://dx.doi.org/10.1088/1367-2630/adb44d>.

- [Geo+25b] Timothy N. Georges, Marius Bothe, Christoph Sünderhauf, Bjorn K. Berntson, Róbert Izsák, and Aleksei V. Ivanov. “Quantum simulations of chemistry in first quantization with any basis set”. In: *npj Quantum Information* 11.1 (Apr. 2025). ISSN: 2056-6387. DOI: [10.1038/s41534-025-00987-1](https://doi.org/10.1038/s41534-025-00987-1). URL: <http://dx.doi.org/10.1038/s41534-025-00987-1>.
- [Gij09] Dion Gijswijt. *Block diagonalization for algebra’s associated with block codes*. 2009. arXiv: [0910.4515](https://arxiv.org/abs/0910.4515) [math.OC]. URL: <https://arxiv.org/abs/0910.4515>.
- [Gri25] Dmitry Grinko. “Mixed Schur-Weyl duality in quantum information”. PhD thesis. Universiteit van Amsterdam, Feb. 2025. URL: <http://hdl.handle.net/11245.1/d9e16c26-ef20-40c5-b847-53c13d1a8a1a>.
- [Haa+17] Jeongwan Haah, Aram W. Harrow, Zhengfeng Ji, Xiaodi Wu, and Nengkun Yu. “Sample-optimal tomography of quantum states”. In: *IEEE Transactions on Information Theory* (2017). ISSN: 1557-9654. DOI: [10.1109/tit.2017.2719044](https://doi.org/10.1109/TIT.2017.2719044). URL: <http://dx.doi.org/10.1109/TIT.2017.2719044>.
- [Had+22] Charles Hadfield, Sergey Bravyi, Rudy Raymond, and Antonio Mezzacapo. “Measurements of Quantum Hamiltonians with Locally-Biased Classical Shadows”. In: *Communications in Mathematical Physics* 391 (2022), pp. 951–967. DOI: [10.1007/s00220-022-04343-8](https://doi.org/10.1007/s00220-022-04343-8). URL: <https://doi.org/10.1007/s00220-022-04343-8>.
- [Har+20] Charles R. Harris et al. “Array programming with NumPy”. In: *Nature* 585.7825 (Sept. 2020), pp. 357–362. DOI: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2). URL: <https://doi.org/10.1038/s41586-020-2649-2>.
- [HKP20] Hsin-Yuan Huang, Richard Kueng, and John Preskill. “Predicting many properties of a quantum system from very few measurements”. In: *Nature Physics* 16.10 (June 2020), pp. 1050–1057. ISSN: 1745-2481. DOI: [10.1038/s41567-020-0932-7](https://doi.org/10.1038/s41567-020-0932-7). URL: <http://dx.doi.org/10.1038/s41567-020-0932-7>.
- [KLC24] Sujay Kazi, Martín Larocca, and M Cerezo. “On the universality of S_n -equivariant k -body gates”. In: *New Journal of Physics* 26.5 (May 2024), p. 053030. ISSN: 1367-2630. DOI: [10.1088/1367-2630/ad4819](https://doi.org/10.1088/1367-2630/ad4819). URL: <http://dx.doi.org/10.1088/1367-2630/ad4819>.
- [LC19] Guang Hao Low and Isaac L. Chuang. “Hamiltonian Simulation by Qubitization”. In: *Quantum* 3 (July 2019), p. 163. ISSN: 2521-327X. DOI: [10.22331/q-2019-07-12-163](https://doi.org/10.22331/q-2019-07-12-163). URL: <http://dx.doi.org/10.22331/q-2019-07-12-163>.
- [LMG65] H.J. Lipkin, N. Meshkov, and A.J. Glick. “Validity of many-body approximation methods for a solvable model: (I). Exact solutions and perturbation theory”. In: *Nuclear Physics* 62.2 (1965), pp. 188–198. ISSN: 0029-5582. DOI: [10.1016/0029-5582\(65\)90862-X](https://www.sciencedirect.com/science/article/pii/002955826590862X). URL: <https://www.sciencedirect.com/science/article/pii/002955826590862X>.
- [Mes06] Roy Meshulam. “An uncertainty inequality for finite abelian groups”. In: *European Journal of Combinatorics* 27.1 (2006), pp. 63–67. ISSN: 0195-6698. DOI: [10.1016/j.ejc.2004.07.009](https://www.sciencedirect.com/science/article/pii/S0195669804001453). URL: <https://www.sciencedirect.com/science/article/pii/S0195669804001453>.
- [Mor73] Jacques Morgenstern. “Note on a Lower Bound on the Linear Complexity of the Fast Fourier Transform”. In: *Journal of the ACM* 20.2 (1973), pp. 305–306. DOI: [10.1145/321752.321761](https://doi.org/10.1145/321752.321761). URL: <https://doi.org/10.1145/321752.321761>.

- [Muñ+18] A. Muñoz, A. B. Klimov, M. Grassl, and L. L. Sánchez-Soto. “Tomography from collective measurements”. In: *Quantum Information Processing* 17.10, 286 (Sept. 2018). ISSN: 1573-1332. DOI: [10.1007/s11128-018-2045-0](https://doi.org/10.1007/s11128-018-2045-0). URL: <http://dx.doi.org/10.1007/s11128-018-2045-0>.
- [Nat] National Institute of Standards and Technology. *NIST Digital Library of Mathematical Functions, Section 18.22: Hahn Class: Recurrence Relations and Differences*. URL: <https://dlmf.nist.gov/18.22> (visited on 09/17/2026).
- [NC00] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 2000.
- [Ope26a] OpenAI. *An explicit power saving for the exact discrete Fourier transform*. OpenAI Math Release preprint [OAI:An-explicit-power-saving-for-the-exact-discrete-Fourier-transform-September-25-2026](https://arxiv.org/abs/2026.09.25). 2026.
- [Ope26b] OpenAI. *An Upper Bound of 9/4 for the Matrix Multiplication Exponent*. OpenAI Math Release preprint [OAI:Matrix-Multiplication-Nine-Fourths-October-2-2026](https://arxiv.org/abs/2026.10.02). 2026.
- [Sag91] Bruce E. Sagan. *The symmetric group - representations, combinatorial algorithms, and symmetric functions*. Wadsworth & Brooks / Cole mathematics series. Wadsworth, 1991, pp. I–XV, 1–197. ISBN: 978-0-534-15540-7.
- [Sau26] Lennart Sauer. *Pauli transforms*. GitHub repository. Commit 8596eb0. Accuracy measurements use source commit 3829b72. 2026. URL: <https://github.com/SourLen/pauli-transforms/tree/8596eb0bc53dbbc11b16be1104c4d713cb2b18bb> (visited on 09/27/2026).
- [SBG26a] Daniel J. Spencer, Kishor Bharti, and Alexey V. Gorshkov. *An efficient Pauli decomposition algorithm for structured matrices*. 2026. arXiv: [2606.31952](https://arxiv.org/abs/2606.31952) [quant-ph]. URL: <https://arxiv.org/abs/2606.31952>.
- [SBG26b] Daniel J. Spencer, Kishor Bharti, and Alexey V. Gorshkov. *An efficient Pauli decomposition algorithm for structured matrices*. 2026. URL: <https://github.com/dspencer2596/sparse-pauli-decomposition>.
- [Sch+24] Louis Schatzki, Martín Larocca, Quynh T. Nguyen, Frédéric Sauvage, and M. Cerezo. “Theoretical guarantees for permutation-equivariant quantum neural networks”. In: *npj Quantum Information* 10.1 (Jan. 2024). ISSN: 2056-6387. DOI: [10.1038/s41534-024-00804-1](https://doi.org/10.1038/s41534-024-00804-1). URL: <http://dx.doi.org/10.1038/s41534-024-00804-1>.
- [Sha+18] Nathan Shammah, Shahnawaz Ahmed, Neill Lambert, Simone De Liberato, and Franco Nori. “Open quantum systems with local and collective incoherent processes: Efficient numerical simulations using permutational invariance”. In: *Physical Review A* 98.6 (2018), p. 063815. DOI: [10.1103/PhysRevA.98.063815](https://doi.org/10.1103/PhysRevA.98.063815).
- [Ski20] Steven S. Skiena. *The Algorithm Design Manual*. 3rd ed. Springer, 2020. DOI: [10.1007/978-3-030-54256-6](https://doi.org/10.1007/978-3-030-54256-6).
- [Sob25] Aleksandros Sobczyk. “Deterministic Complexity Analysis of Hermitian Eigenproblems”. In: *52nd International Colloquium on Automata, Languages, and Programming (ICALP 2025)*. Vol. 334. Leibniz International Proceedings in Informatics (LIPIcs). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025, 131:1–131:21. DOI: [10.4230/LIPIcs.ICALP.2025.131](https://doi.org/10.4230/LIPIcs.ICALP.2025.131). URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ICALP.2025.131>.

- [Str69] Volker Strassen. “Gaussian elimination is not optimal”. In: *Numerische Mathematik* 13 (1969), pp. 354–356. DOI: [10.1007/BF02165411](https://doi.org/10.1007/BF02165411). URL: <https://api.semanticscholar.org/CorpusID:121656251>.
- [Tót+10] Géza Tóth, Witlef Wieczorek, David Gross, Roland Krischek, Christian Schwemmer, and Harald Weinfurter. “Permutationally Invariant Quantum Tomography”. In: *Physical Review Letters* 105.25 (Dec. 2010), p. 250403. DOI: [10.1103/PhysRevLett.105.250403](https://doi.org/10.1103/PhysRevLett.105.250403). URL: <https://doi.org/10.1103/PhysRevLett.105.250403>.
- [Val09] Frank Vallentin. “Symmetry in semidefinite programs”. In: *Linear Algebra and its Applications* 430.1 (Jan. 2009), pp. 360–369. ISSN: 0024-3795. DOI: [10.1016/j.laa.2008.07.025](https://doi.org/10.1016/j.laa.2008.07.025). URL: <http://dx.doi.org/10.1016/j.laa.2008.07.025>.
- [Van00] Charles F. Van Loan. “The ubiquitous Kronecker product”. In: *Journal of Computational and Applied Mathematics* 123.1 (2000). Numerical Analysis 2000. Vol. III: Linear Algebra, pp. 85–100. ISSN: 0377-0427. DOI: [10.1016/S0377-0427\(00\)00393-9](https://doi.org/10.1016/S0377-0427(00)00393-9). URL: <https://www.sciencedirect.com/science/article/pii/S0377042700003939>.
- [WBC22] Kianna Wan, Mario Berta, and Earl T. Campbell. “Randomized Quantum Algorithm for Statistical Phase Estimation”. In: *Physical Review Letters* 129.3 (July 2022), p. 030503. DOI: [10.1103/PhysRevLett.129.030503](https://doi.org/10.1103/PhysRevLett.129.030503). URL: <https://doi.org/10.1103/PhysRevLett.129.030503>.
- [WMB24] Samson Wang, Sam McArdle, and Mario Berta. “Qubit-Efficient Randomized Quantum Algorithms for Linear Algebra”. In: *PRX Quantum* 5.2 (Apr. 2024), p. 020324. DOI: [10.1103/PRXQuantum.5.020324](https://doi.org/10.1103/PRXQuantum.5.020324). URL: <https://doi.org/10.1103/PRXQuantum.5.020324>.
- [YH96] R. K. Rao Yarlagadda and John E. Hershey. *Hadamard matrix analysis and synthesis: with applications to communications and signal/image processing*. Springer, 1996. DOI: [10.1007/978-1-4615-6313-6](https://doi.org/10.1007/978-1-4615-6313-6). URL: <https://doi.org/10.1007/978-1-4615-6313-6>.

Appendix A

Group and representation theory

A.1 Group orbits

Proof of Proposition 2.19. Let e be the identity of G . Since $ex = x$ for every $x \in X$, the relation is reflexive. If $y = gx$, then $x = g^{-1}y$, so it is symmetric. If $y = gx$ and $z = hy$, then $z = (hg)x$, so it is transitive. Thus $\sim$ is an equivalence relation, and the equivalence class of x is precisely its orbit $G \cdot x$. As is well known, equivalence classes partition sets. $\square$

A.2 Basic representation theory concepts and proofs

We assume familiarity with basic group theory and linear algebra. All vector spaces are finite dimensional unless stated otherwise.

Definition A.1 (Representation). Let V be a complex vector space, and let $\text{GL}(V)$ be the group of invertible linear maps $V \rightarrow V$. A *representation* of a group G on V is a group homomorphism $\rho : G \rightarrow \text{GL}(V)$, that is,

$$\rho(gh) = \rho(g)\rho(h), \quad \rho(e) = I_V, \quad g, h \in G. \quad (\text{A.1})$$

If V is equipped with an inner product and every $\rho(g)$ is unitary, the representation is called *unitary*.

The homomorphism property also gives $\rho(g^{-1}) = \rho(g)^{-1}$. We sometimes refer to V itself as the representation when the action is understood. The unitary group

$$U(2) := \{U \in \mathbb{C}^{2 \times 2} : U^\dagger U = I_2\}$$

consists of all unitary operators on a single qubit, with matrix multiplication as the group operation. This group is infinite. The maps $\pi \mapsto U_\pi$ and $U \mapsto U^{\otimes n}$ on $\mathcal{H}_n$ are unitary representations of S_n and $U(2)$, respectively.

Definition A.2 (Invariant subspace and irreducibility). A subspace $W \subseteq V$ is *invariant* under a representation ρ if $\rho(g)W \subseteq W$ for every $g \in G$. The restrictions $\rho(g)|_W$ then define a representation on W , called a *subrepresentation*. A representation on a nonzero space V is *irreducible* if its only invariant subspaces are $\{0\}$ and V , so it only has trivial subrepresentations.

The condition $\rho(g)W \subseteq W$ is equivalent to $\rho(g)W = W$, since it also has to hold for g^{-1} . A vector v is called invariant if $\rho(g)v = v$ for every $g \in G$.

Representations ρ on V and σ on W induce representations on the direct sum and tensor product by

$$g \mapsto \rho(g) \oplus \sigma(g), \quad g \mapsto \rho(g) \otimes \sigma(g), \quad (\text{A.2})$$

respectively. In particular, if G acts trivially¹ on a space M , its action on $M \otimes V$ is $I_M \otimes \rho(g)$. Choosing a basis of M identifies this representation with $\dim M$ copies of V . We call M a *multiplicity space*.

Definition A.3 (*G-homomorphism and equivalent representations*). Let ρ and σ be representations of the same group G on two vector spaces V and W . A linear map $T : V \rightarrow W$ is a *G-homomorphism* if

$$T\rho(g) = \sigma(g)T \quad \text{for every } g \in G. \quad (\text{A.3})$$

Two representations are *equivalent*, or *isomorphic*, if there is an invertible G -homomorphism between them.

When $V = W$ and $\rho = \sigma$, a G -homomorphism is an operator commuting with every $\rho(g)$.

Lemma A.4 (Schur's lemma). *Let ρ and σ be irreducible representations of a group G on nonzero finite-dimensional complex vector spaces V and W . Every G -homomorphism $T : V \rightarrow W$ is either zero or an isomorphism. In particular, it is zero when the representations are inequivalent. If $V = W$ and $\rho = \sigma$, then $T = cI_V$ for some $c \in \mathbb{C}$.*

Proof. Both $\ker T$ and $\text{im } T$ are invariant subspaces. Indeed, if $v \in \ker T$, then $T\rho(g)v = \sigma(g)Tv = 0$, and if $w = Tv$, then $\sigma(g)w = T\rho(g)v \in \text{im } T$. Suppose $T \neq 0$. Irreducibility gives $\ker T = \{0\}$ and $\text{im } T = W$, so T is invertible. Its inverse is also a G -homomorphism, as follows by multiplying (A.3) on both sides by T^{-1} .

Now let $T : V \rightarrow V$ commute with $\rho(g)$ for every $g \in G$. Since V is nonzero and finite-dimensional over $\mathbb{C}$, T has an eigenvalue $c \in \mathbb{C}$. The map $T - cI_V$ is again a G -homomorphism, and its kernel contains an eigenvector. By the first part, this map must be zero. Hence $T = cI_V$. $\square$

See also [FH13, Lemma 1.7].

A.3 The permutation representation

Proof of Corollary 2.13. Let $\pi, \sigma \in S_n$ be two permutations. Then for any basis vector $|i_1\rangle \otimes \cdots \otimes |i_n\rangle \in \mathcal{H}_n$, we have

$$\pi(\sigma(|i_1\rangle \otimes \cdots \otimes |i_n\rangle)) = \pi(|i_{\sigma^{-1}(1)}\rangle \otimes \cdots \otimes |i_{\sigma^{-1}(n)}\rangle) \quad (\text{A.4})$$

$$= |i_{(\pi\sigma)^{-1}(1)}\rangle \otimes \cdots \otimes |i_{(\pi\sigma)^{-1}(n)}\rangle \quad (\text{A.5})$$

$$= (\pi\sigma)(|i_1\rangle \otimes \cdots \otimes |i_n\rangle), \quad (\text{A.6})$$

¹ $\rho(g) = I_M$ defines a representation of every group on an arbitrary vector space M , called the trivial representation.

so the action is a group homomorphism. To prove unitarity, take $\pi \in S_n$ and two basis vectors $|i_1\rangle \otimes \cdots \otimes |i_n\rangle$ and $|j_1\rangle \otimes \cdots \otimes |j_n\rangle$. Then

$$\langle \pi(|i_1\rangle \otimes \cdots \otimes |i_n\rangle), \pi(|j_1\rangle \otimes \cdots \otimes |j_n\rangle) \rangle \quad (\text{A.7})$$

$$= \prod_{k=1}^n \langle |i_{\pi^{-1}(k)}\rangle, |j_{\pi^{-1}(k)}\rangle \rangle \quad (\text{A.8})$$

$$= \prod_{k=1}^n \delta_{i_k, j_k} \quad (\text{A.9})$$

$$= \langle |i_1\rangle \otimes \cdots \otimes |i_n\rangle, |j_1\rangle \otimes \cdots \otimes |j_n\rangle \rangle, \quad (\text{A.10})$$

where we used orthonormality of the computational basis. $\square$

A.4 Proving the invariant block form

The actions $U \mapsto U^{\otimes n}$ of the unitary group $U(2)$ and $\pi \mapsto U_\pi$ of S_n commute. Schur-Weyl duality describes their simultaneous decomposition. We state the result for qubits. A general treatment and proof can be found in [Gri25, Section 2.10] or [FH13, Section 6].

Theorem A.5 (Schur-Weyl duality). *There is a unitary isomorphism*

$$\mathcal{H}_n \cong (\mathbb{C}^2)^{\otimes n} \cong \bigoplus_{\substack{\lambda \vdash n \\ \ell(\lambda) \leq 2}} \mathcal{W}_\lambda \otimes \mathcal{S}_\lambda, \quad (\text{A.11})$$

where $\mathcal{W}_\lambda$ is an irreducible representation q_λ of $U(2)$ and $\mathcal{S}_\lambda$ is an irreducible representation ρ_λ of S_n . Representations for different λ are inequivalent. Under this identification,

$$U^{\otimes n} \cong \bigoplus_{\lambda} q_\lambda(U) \otimes I_{\mathcal{S}_\lambda}, \quad U \in U(2), \quad (\text{A.12})$$

$$U_\pi \cong \bigoplus_{\lambda} I_{\mathcal{W}_\lambda} \otimes \rho_\lambda(\pi), \quad \pi \in S_n. \quad (\text{A.13})$$

Here $\lambda \vdash n$ denotes a partition of n , a non-increasing sequence of positive integers summing to n , and $\ell(\lambda)$ is the number of its parts. The partitions with at most two parts are $\lambda = (n-k, k)$ for $0 \leq k \leq \lfloor n/2 \rfloor$. As in Equation (2.39) we set

$$\mathcal{W}_k := \mathcal{W}_{(n-k, k)}, \quad \mathcal{S}_k := \mathcal{S}_{(n-k, k)}, \quad (\text{A.14})$$

with dimensions given in Equation (2.40). Viewed as a representation of S_n , the summand $\mathcal{W}_k \otimes \mathcal{S}_k$ consists of d_k equivalent copies of $\mathcal{S}_k$, with $\mathcal{W}_k$ as the multiplicity space.

Proof of Corollary 2.26. Permutation invariance is equivalent to $AU_\pi = U_\pi A$ for every $\pi \in S_n$. After choosing bases of the spaces $\mathcal{W}_k$, each block of A between copies of $\mathcal{S}_k$ and $\mathcal{S}_{k'}$ is an S_n -homomorphism. By Lemma A.4, it vanishes for $k \neq k'$ and is a scalar multiple of the identity for $k = k'$. Conversely, every operator of the form in Equation (2.41) commutes with the permutation action in Equation (A.13). Multiplication and conjugate transpose act separately on the blocks, proving the $*$ -algebra isomorphism in Equation (2.42). $\square$

Remark A.6. It should be noted that to prove the block decomposition of permutation-invariant operators, we do not need the full result of Schur-Weyl duality. Decomposing the S_n representation into a direct sum of inequivalent irreducible representations via Maschke's Theorem and applying Schur's lemma would yield the same result. We include the full Schur-Weyl duality result here for completeness and because we believe it to supply a deeper insight into the structure of the problem.

A.5 Proof of Schur basis properties

We identify the vectors $|u_{k,r}\rangle$ in Equation (5.6) with the normalized vectors in Gijswijt's block diagonalization and use his results to establish orthonormality.

Proof. Let $e_{k,r}$ be Gijswijt's vector for the tableau of shape $(n-k, k)$ described in [Gij09, Section 5]. After relabeling his symbols 1, 2 as 0, 1, this tableau has 0 above 1 in each of its first k columns and $r-k$ ones among the remaining $m := n-2k$ boxes in the first row. All other entries are zero. Let π_k reorder the tensor factors by reading each two-box column from top to bottom, followed by the remaining first-row boxes. This permutation depends on k but not on r .

Gijswijt's Equation (13) defines $e_{k,r}$ by summing over distinct permutations of the entries within each row, then summing over column swaps with a minus sign for each swap. A term with 1 in both boxes of a column cancels under that column swap. Hence only row arrangements with 0 above 1 in all two-box columns contribute. Their remaining boxes contain $r-k$ ones. Each column contributes $|01\rangle - |10\rangle$, so

$$U_{\pi_k} e_{k,r} = (|01\rangle - |10\rangle)^{\otimes k} \otimes \sum_{\substack{v \in \{0,1\}^m \\ |v|=r-k}} |v\rangle.$$

To compare this with Equation (5.6), write $|\psi^-\rangle = (|01\rangle - |10\rangle)/\sqrt{2}$. On each such factor of $|u_{k,k}\rangle$, the two corresponding terms in S_- cancel as $(|11\rangle - |11\rangle)/\sqrt{2} = 0$. On the last m qubits, initially in $|0\rangle^{\otimes m}$, each string of weight $r-k$ is obtained by flipping the positions where that string has a 1, in $(r-k)!$ orders. With the normalization in Equation (5.6), this gives

$$\begin{aligned} |u_{k,r}\rangle &= \frac{U_{\pi_k} e_{k,r}}{\sqrt{2^k \binom{m}{r-k}}} \\ &= |\psi^-\rangle^{\otimes k} \otimes \frac{1}{\sqrt{\binom{m}{r-k}}} \sum_{\substack{v \in \{0,1\}^m \\ |v|=r-k}} |v\rangle. \end{aligned} \tag{A.15}$$

□

For fixed k , Gijswijt establishes that the $e_{k,r}$ are orthogonal and have squared norm $2^k \binom{m}{r-k}$ [Gij09, Section 5, Equation (53)]. Thus the $|u_{k,r}\rangle$ are orthonormal for fixed k . For different k , the vectors belong to mutually orthogonal sums of irreducible S_n -representations of types $(n-k, k)$ [Gij09, Sections 2.2 and 3.1], and are therefore orthogonal for distinct k . The explicit expression in Equation (A.15) shows that every computational basis string has k ones in the first $2k$ positions and $r-k$ in the remaining positions, giving the total weight r .

For fixed k , we obtain the vectors $|u_{k,r}\rangle$ from Gijswijt's normalized representatives by applying the same permutation U_{π_k} for every r . Every permutation-invariant operator commutes with U_{π_k} , so this leaves its matrix elements unchanged. The vectors $|u_{k,r}\rangle$ therefore give the same Schur blocks as Gijswijt's representatives [Gij09, Theorem 6 and Equation (54)], and we can take $|u_{k,r}\rangle = |k, r, 1\rangle$.

The operator B_{rs}^t maps the weight- s subspace into the weight- r subspace and vanishes on all other weight subspaces. Since $|u_{k,r}\rangle$ has weight r , each Schur block of B_{rs}^t can have a nonzero entry only in row r and column s . The corresponding basis vectors $|u_{k,r}\rangle$ and $|u_{k,s}\rangle$ exist only when $k \leq r, s \leq n-k$, or equivalently $k \leq \min(r, s, n-r, n-s)$.

A.6 Spectral consequences of the Schur decomposition

We derive the eigenbasis formula and operation counts used in [Section 6.1](#).

Let H be a permutation-invariant Hermitian operator with Schur blocks H_k . Choose an orthonormal eigenbasis of each block, $H_k|v_{k,j}\rangle = E_{k,j}|v_{k,j}\rangle$. Since H acts as $H_k \otimes I_{\mu_k}$ on the k -th summand, we have

$$H(|v_{k,j}\rangle \otimes |a\rangle_k) = E_{k,j}(|v_{k,j}\rangle \otimes |a\rangle_k). \quad (\text{A.16})$$

The vectors $|v_{k,j}\rangle \otimes |a\rangle_k$ are orthonormal within each summand, and distinct summands are orthogonal. There are $\sum_k d_k \mu_k = 2^n$ such vectors, so they form the complete eigenbasis in [Equation \(6.1\)](#). Each block eigenvector gives μ_k orthogonal eigenstates with eigenvalue $E_{k,j}$. Choosing $a = 1$ gives a representative eigenstate expressed in the vectors $|k, r, 1\rangle = |u_{k,r}\rangle$ from [Equation \(5.6\)](#).

Each block has d_k eigenvalues, counted with multiplicity, and its orthonormal eigenbasis is specified by d_k^2 coefficients. Summing over $0 \leq k \leq \lfloor n/2 \rfloor$ gives

$$\sum_k d_k = \left\lfloor \frac{(n+2)^2}{4} \right\rfloor, \quad \sum_k d_k^2 = \binom{n+3}{3}. \quad (\text{A.17})$$

There are $O(n)$ blocks, each of dimension at most $n+1$. An eigensolver requiring $O(d_k^3)$ operations for block H_k therefore costs $\sum_k O(d_k^3) = O(n^4)$ in total, as used in [Equation \(6.2\)](#).

To construct a single block H_k from matrix-unit orbit coefficients, consider $K_{k;rs}^t$ in [Proposition 5.2](#). The factor $\binom{k}{s-t-l}$ vanishes unless $0 \leq s-t-l \leq k$, so the sum has at most $k+1$ nonzero terms. After tabulating the binomial coefficients in $O(n^2)$ operations, each $K_{k;rs}^t$ can therefore be computed in $O(k+1)$ operations. There are $O(n^2)$ pairs (r, s) and $O(n)$ values of t per pair. Computing these coefficients and summing $(H_k)_{r,s} = \sum_t d_{rs}^t K_{k;rs}^t$ therefore costs $O((k+1)n^3)$ operations. This is $O(n^3)$ when k is held constant as n grows.

A.7 Other symmetry groups

For any subgroup of S_n , the matrix-unit and Pauli orbit sums form bases of the invariant operators by [Proposition 2.20](#). To obtain the corresponding blocks, we decompose the Hilbert space into irreducible representations of the subgroup and apply Schur's lemma. We consider two examples below.

A.7.1 Separate permutations of two sets

Divide the qubits into two sets of sizes n_1 and n_2 , with $n_1 + n_2 = n$, and let $G = S_{n_1} \times S_{n_2}$ permute the qubits within each set. Imposing symmetry over the two groups separately gives

$$\text{End}(\mathcal{H}_n)^G \cong \text{End}(\mathcal{H}_{n_1})^{S_{n_1}} \otimes \text{End}(\mathcal{H}_{n_2})^{S_{n_2}}. \quad (\text{A.18})$$

The orbit labels record the four local operator counts in each set. The basis elements are tensor products of the orbit sums from [Section 2.2.2](#), so the number of coordinates is $\binom{n_1+3}{3} \binom{n_2+3}{3}$.

Applying the Schur decomposition to each set gives blocks indexed by (k_1, k_2) that act on $\mathcal{W}_{k_1}^{(n_1)} \otimes \mathcal{W}_{k_2}^{(n_2)}$. Their dimensions are at most $(n_1+1)(n_2+1)$. We can apply the conversions in [Chapters 4](#) and [5](#) to one set index at a time, while the overall arithmetic cost remains polynomial. The same argument applies to any fixed number of sets.

A.7.2 Cyclic translations

Let $C_n = \langle T \rangle$, where $T|i_1 \cdots i_n\rangle = |i_n i_1 \cdots i_{n-1}\rangle$ translates a ring of qubits. The matrix-unit and Pauli orbit sums now consist of distinct cyclic shifts of a tensor product string. As in [Chapter 4](#), the local change of basis $F^{\otimes n}$ commutes with these shifts and therefore restricts to a conversion between the two orbit bases.

The irreducible complex representations of C_n are one-dimensional. Indeed, since the group is abelian, the representing operators commute with one another and hence are scalar by [Lemma A.4](#). Irreducibility then forces the representation space to have dimension one. Since $T^n = I$, these representations map T to the n th roots of unity ω^q , where $\omega = \exp(2\pi i/n)$ and $0 \leq q < n$. Each vector in a basis of the eigenspace with eigenvalue ω^q spans a copy of the one-dimensional representation $T^j \mapsto \omega^{qj}$. The dimension of this eigenspace is therefore the multiplicity of that representation, and the eigenspace serves as its multiplicity space.

Every invariant operator commutes with T by definition and therefore preserves these eigenspaces. Conversely, any operator acting within the eigenspaces commutes with T , since T is scalar on each one. Thus the invariant operators have arbitrary matrix blocks on these eigenspaces. The projector onto the eigenspace with eigenvalue ω^q is

$$\Pi_q = \frac{1}{n} \sum_{j=0}^{n-1} \omega^{-qj} T^j, \quad 0 \leq q < n, \quad (\text{A.19})$$

which gives a Fourier sum of the translations. Applying Π_q to one computational basis state from each translation orbit and normalizing the nonzero results gives an orthonormal basis of the eigenspace.

There are at most n such eigenspaces, and their dimensions sum to 2^n . At least one block therefore has dimension at least $2^n/n$, so cyclic symmetry alone leaves exponentially large blocks, as also discussed in [\[Ans+23, Figure 1\]](#).

We can also combine permutation symmetry with other symmetry groups. For example, one may imagine n "bulks" of qubits, each obeying a local symmetry group G , and then permuting the bulks. Here we may use the more general construction of Gijswijt [\[Gij09, Section 4\]](#). If a finite group G acts unitarily on each local space $\mathbb{C}^d$, the combined symmetry group is the wreath product $G \wr S_n$, with invariant algebra $\text{Sym}^n(\text{End}(\mathbb{C}^d)^G)$. For fixed d , Gijswijt constructs its blocks in polynomial time from a known (promised) block decomposition of the local invariant algebra. Extending our concrete coordinate conversions to these algebras and finding physical applications remain questions for further work.

Appendix B

Sparsity and transformation details

B.1 The Pauli basis

Proof of Lemma 2.3. For $P, Q \in \mathcal{P}_n$, the Hilbert-Schmidt inner product is

$$\langle P, Q \rangle_{HS} = \text{Tr}(P^\dagger Q) = \prod_{i=1}^n \text{Tr}(P_i Q_i) = 2^n \delta_{P,Q}, \quad (\text{B.1})$$

where the second equality follows from Equation (2.1) and Proposition 2.2. Thus the Pauli strings are orthogonal. There are 4^n of them, equal to the dimension of $\text{End}(\mathcal{H}_n)$, so they form a basis. Taking adjoints of the Pauli expansion of a Hermitian operator and using uniqueness of the coefficients shows that the coefficients are real. Hence $\mathcal{P}_n$ also forms a basis of the real vector space of Hermitian operators. $\square$

Remark B.1. Many alternative proofs of this fact exist. For example, one may show that the Pauli matrices form a basis of the space of 2×2 matrices, and then use the fact that the tensor product of two bases is again a basis of the tensor product space, which follows immediately from the definition of the tensor product.

B.2 Pauli row-sparsity and computational sparsity

The bound in Theorem 2.9 follows directly from the binary representation of Pauli strings: the label x determines the positions of their non-zero entries, while z only affects their phases.

Proof of Theorem 2.9. All indices below are binary strings in $\{0, 1\}^n$. Let

$$R := \{x : \exists z \text{ with } \alpha_{x,z} \neq 0\}. \quad (\text{B.2})$$

Thus $|R|$ is the number of non-zero rows of the Pauli coefficient matrix. Since $X|q_j\rangle = |q_j \oplus 1\rangle$ and $Z|q_j\rangle = (-1)^{q_j}|q_j\rangle$, the convention of Proposition 2.6 gives

$$\begin{aligned} P(x, z)|q\rangle &= (-i)^{|x \wedge z|} (-1)^{|z \wedge (q \oplus x)|} |q \oplus x\rangle \\ &= i^{|x \wedge z|} (-1)^{|z \wedge q|} |q \oplus x\rangle. \end{aligned} \quad (\text{B.3})$$

The second equality uses $|z \wedge (q \oplus x)| = |z \wedge q| + |z \wedge x| \pmod{2}$. Taking matrix elements in the Pauli expansion of A therefore yields

$$\langle p|A|q\rangle = \sum_z \alpha_{p \oplus q, z} i^{|(p \oplus q) \wedge z|} (-1)^{|z \wedge q|}, \quad (\text{B.4})$$

because $p = q \oplus x$ holds precisely when $x = p \oplus q$. In particular,

$$\langle p|A|q\rangle \neq 0 \implies p \oplus q \in R. \quad (\text{B.5})$$

For a fixed column q , every non-zero entry thus has its row index in $\{q \oplus x : x \in R\}$. The map $x \mapsto q \oplus x$ is a bijection, so this set has $|R|$ elements. Likewise, for a fixed row p , every non-zero entry has its column index in $\{p \oplus x : x \in R\}$, again a set of $|R|$ elements. Hence every row and every column contains at most $|R|$ non-zero entries, proving the claim. $\square$

B.3 A factorization of the Schur conversion coefficients

We derive Equation (5.14) directly from the alternative formula for the matrix elements in [Gij09, Equations (54) and (56)]. In our normalization, that formula reads

$$K_{k;rs}^t = \frac{\sum_{u=\max(k,t)}^{\min(r,s)} (-1)^{u-t} \binom{u}{t} \binom{n-2k}{u-k} \binom{n-k-u}{r-u} \binom{n-k-u}{s-u}}{\sqrt{\binom{n-2k}{r-k} \binom{n-2k}{s-k}}}. \quad (\text{B.6})$$

Gijswijt identifies this expression with the coefficient used in Proposition 5.2. Expanding the three binomial coefficients that depend on k gives

$$\begin{aligned} & \binom{n-2k}{u-k} \binom{n-k-u}{r-u} \binom{n-k-u}{s-u} \\ &= \frac{(n-2k)!(n-k-u)!}{(u-k)!(r-u)!(s-u)!(n-k-r)!(n-k-s)!}. \end{aligned} \quad (\text{B.7})$$

Dividing this expression by the square-root normalization in Equation (B.6) gives

$$\gamma_{k;rs} \frac{(n-k-u)!}{(u-k)!(r-u)!(s-u)!}. \quad (\text{B.8})$$

Substituting back into the sum and using Equation (5.12) proves Equation (5.14). Terms with $u < k$ or $u < t$ vanish by the definitions of F and G , so the sum may start at $u = 0$.

Appendix C

Algorithms and numerical comparisons

C.1 Pseudocode for the coordinate transformations

The following algorithms act on common coefficients of the unnormalized orbit sums.

C.1.1 Matrix-unit and Pauli orbit coefficients

We use the tables $M^{(w)}$ and $C^{(w)}$ from [Equation \(4.18\)](#), both of shape $(n - w + 1) \times (w + 1)$. We first construct the Krawtchouk matrices using the recurrence in [Equation \(4.24\)](#).

Preparation: Krawtchouk matrices

```
for  $m = 0, \dots, n$ :  
  Allocate  $Q_m$  of shape  $(m + 1) \times (m + 1)$ .  
  for  $g = 0, \dots, m$ :  
     $[Q_m]_{g,0} \leftarrow 1$ .  
    if  $m \geq 1$ :  $[Q_m]_{g,1} \leftarrow m - 2g$ .  
    for  $h = 1, \dots, m - 1$ :  
       $[Q_m]_{g,h+1} \leftarrow \frac{(m - 2g)[Q_m]_{g,h} - (m - h + 1)[Q_m]_{g,h-1}}{h + 1}$ .  
return  $Q_0, \dots, Q_n$ .
```

Conversion: matrix-unit orbits $\leftrightarrow$ Pauli orbits

```

Prepare  $Q_0, \dots, Q_n$  unless supplied
for  $w = 0, \dots, n$ :
    if converting matrix-unit to Pauli orbits:
        Get  $M^{(w)}$  from the input array
         $T \leftarrow 2^{-w} M^{(w)} Q_w^T$ 
         $C^{(w)} \leftarrow 2^{-(n-w)} Q_{n-w} T$ 
         $C_{:,g}^{(w)} \leftarrow (-i)^g C_{:,g}^{(w)}$  for  $g = 0, \dots, w$ 
        Load  $C^{(w)}$  into the output array
    else, converting Pauli to matrix-unit coefficients:
        Get  $C^{(w)}$  from the input array
         $T_{:,g} \leftarrow i^g C_{:,g}^{(w)}$  for  $g = 0, \dots, w$ 
         $U \leftarrow Q_{n-w} T$ ;  $M^{(w)} \leftarrow U Q_w^T$ 
        Load  $M^{(w)}$  into the output array
return the output array.

```

Here $T_{:,g}$ denotes column g .

C.1.2 Matrix-unit orbit coefficients and Schur blocks

The second algorithm evaluates the factorization in [Equation \(5.15\)](#). The labels of H_k are the weights $k, \dots, n-k$, and each matrix below has shape $(n+1) \times (n+1)$. Write $\kappa_{rs} = \min(r, s, n-r, n-s)$.

Preparation. Generate $0!, \dots, n!$ successively and the binomial coefficients by Pascal's rule. Construct F and G from [Equation \(5.12\)](#), setting entries outside the stated ranges to zero, and compute $\mu_k = \binom{n}{k} - \binom{n}{k-1}$. The factorial table gives the remaining weights when needed:

$$\gamma_{k;rs} = \sqrt{\frac{(r-k)!(s-k)!}{(n-k-r)!(n-k-s)!}}, \quad v_{rs}^t = \frac{n!}{(n-r-s+t)!(r-t)!(s-t)!t!}. \quad (\text{C.1})$$

Conversion: matrix-unit orbits $\leftrightarrow$ Schur blocks

```

Prepare the shared factors as above, unless supplied
for  $r = 0, \dots, n$ :
     $X \leftarrow 0_{(n+1) \times (n+1)}$ 
    if converting matrix-units to Schur blocks:
         $X_{ts} \leftarrow a_{rs}^t$  for  $0 \leq s \leq n, \max(0, r+s-n) \leq t \leq \min(r, s)$ 
         $Y \leftarrow GX$ 
    else converting Schur blocks to matrix-units:
         $X_{ks} \leftarrow \mu_k \gamma_{k;rs} (H_k)_{r,s}$  for  $0 \leq s \leq n, 0 \leq k \leq \kappa_{rs}$ 
         $Y \leftarrow F^T X$ 
    for  $u, s = 0, \dots, n$ :
        if  $u \leq \min(r, s)$ :  $Y_{us} \leftarrow Y_{us} / ((r-u)!(s-u)!)$ 
        else:  $Y_{us} \leftarrow 0$ 
    if converting matrix-units to Schur blocks:
         $Z \leftarrow FY$ .
         $(H_k)_{r,s} \leftarrow \gamma_{k;rs} Z_{ks}$  for  $0 \leq s \leq n, 0 \leq k \leq \kappa_{rs}$ 
    else:
         $Z \leftarrow G^T Y$ 
         $a_{rs}^t \leftarrow Z_{ts} / v_{rs}^t$  for  $0 \leq s \leq n,$ 
             $\max(0, r+s-n) \leq t \leq \min(r, s)$ .
return the blocks  $H_k$  or the coefficients  $a_{rs}^t$ .

```

The two algorithms above require at most $O(n^3)$ preprocessing operations and $O(n^4)$ operations in total with ordinary matrix multiplication. Replacing the matrix products by multiplication with exponent τ gives $O(n^{1+\tau})$, as proved in [Theorems 4.3](#) and [5.4](#). The Pauli-Schur conversion algorithm is their composition, with the relabeling

$$M_{t,s-t}^{(r+s-2t)} = a_{rs}^t. \quad (\text{C.2})$$

C.1.3 Remarks on the Python implementation

The implementation uses NumPy arrays [[Har+20](#)] and matrix multiplication through the `@` Python operator. The default tables (Krawtchouk, Hahn, ...) are `float64` arrays, while the operator coefficients and output blocks use `complex128`. The default Hahn recurrence backend is described in [Remark 5.5](#). It requires $O(n^4)$ operations to prepare all transform matrices and $O(n^4)$ operations for each conversion. Accuracy checks are summarized in [Section C.2.5](#).

C.2 Implementation and numerical comparisons

C.2.1 Implementation

For the Pauli-to-Schur and fixed-locality comparisons, we use the Krawtchouk and Hahn recurrences described in [Section C.1.3](#) and [remark 5.5](#). The optimized Anschuetz implementation evaluates the combinatorial formula in [[Ans+23](#), Appendix E, Eqs. (70)–(71)], using NumPy array operations and reusing combinatorial factors where possible. For the general conversion, we also compare with the authors' public code [[ABK23](#)] to give an idea of the runtime reduction from our implementation changes.

For general conversion, preparation in our routine constructs the Krawtchouk and Hahn transform matrices for the given n . A "cached" call then applies both transformations,

as described above. The optimized Anschuetz routine instead stores the complete Schur blocks of individual Pauli orbits. A cached call then only forms their linear combination with the input coefficients, which is the main source for efficiency gains in the fixed-locality comparison as we will see below. For fixed-weight inputs, we also include another preprocessing standard, which constructs the Schur images of all active Pauli orbits for all algorithms.

The calculations are performed on one thread on an AMD Ryzen 7 7735U. For the comparisons, at each size and input family we use three inputs, with seven timed repetitions per input after one warmup. We report medians and interquartile ranges in our plots. For each first-use repetition, the tables are rebuilt inside the timed interval. For cached repetitions, they are prepared once before timing and reused. Both timings include input formatting and output construction, but exclude initial imports, input generation and error checks. For a computed operator $\tilde{A}$ and reference A , relative errors mean $\|\tilde{A} - A\|_{HS} / \|A\|_{HS}$. We decided on a 10^{-8} threshold for relative errors in most timing comparisons, but obviously this is chosen rather arbitrarily and depends on the application. Evaluation and random input generation are available in the GitHub repository [Sau26].

C.2.2 General coordinate conversion

Figure 3.1 compares the three routines on the same random Pauli-orbit inputs, with all distinct Schur blocks as the output. The inputs are not restricted further, so this comparison tests the general conversion problem addressed by our complexity bounds. For this comparison, optimized Anschuetz preparation constructs the full bank of Pauli-orbit columns, while our routine prepares the Krawtchouk and Hahn tables described above. The public implementation did not provide the option to cache preparation, so we report only its first-use timings. Optimization substantially reduces the cost of the public code, while our conversion remains faster throughout the tested range when preparation is included. With cached tables, the optimized routine is faster for the smallest systems, but our method becomes faster as the system size increases. The public code is included only up to $n = 5$ because of time constraints, while the other two methods are measured through $n = 40$.

Relative errors in these comparisons remain below 2×10^{-14} , with or without preparation. Independent checks of both directions, including representative inputs through $n = 40$, are given in Section C.2.5.

C.2.3 Matrix-unit orbit to Schur conversion

We compare the *shared-factor* implementation presented in Section 5.2 and the Hahn recurrence in Remark 5.5 with the native conversion of `permqit` [BP26b]. All three routines receive the same random matrix-unit orbit coefficients a_{rs}^t and return every distinct Schur block H_k .

For `permqit`, we use its Gijswijt construction and algebra isomorphism, including the Gram-matrix normalization to an orthonormal block basis. Preprocessing constructs their conversion matrix and the input and output index maps, while a cached call orders the coefficients, applies the matrix accordingly and assembles the blocks. The *shared-factor* implementation retains the factors, factorials and normalization data. The Hahn implementation stores all distinct Hahn transform matrices. Both routines perform the matrix products and apply the normalization factors on every call.

We use $n \in \{2, 4, 6, 8, 10, 12, 16, 20\}$ and five inputs per size. Each first-use trial runs in a fresh process and includes preprocessing and one conversion. The cached timing is again the median of seven further conversions using the same prepared data.

In [Figure 3.2](#), preprocessing increasingly dominates the cost of `permqit`. Our shared-factor implementation has lower first-use runtime than `permqit` throughout the tested range, while cached `permqit` conversion is close to the Hahn implementation and faster than the shared-factor implementation at $n = 20$.

In all 120 trials, the relative Hilbert–Schmidt error was below 10^{-8} .

We use the Hahn recurrence as the default for the Pauli-to-Schur and application benchmarks because the factorial implementation loses accuracy through cancellation at larger n in our precision checks [[Sau26](#)]. At $n = 20$, the Hahn implementation also has a lower cached conversion cost, although preparing its transform matrices is more expensive.

C.2.4 Fixed-locality comparison

The inputs in [Figure C.1](#) are random linear combinations of all Pauli-orbit averages of weight $\ell = 2, 4, 6, 8$. The tested sizes range from $n = 4$ to 40 for $\ell = 2, 4$ and from $n = 8$ to 40 for $\ell = 6, 8$. We compare our general conversion with the fixed-locality formula of Chang et al. [[CLC26](#)] and the optimized Anschuetz implementation.

Chang’s implementation precomputes the necessary binomial factors, then evaluates the different orbit’s contributions and afterwards constructs the sparse output directly. For Anschuetz, preparation constructs the columns corresponding to the active Pauli orbits. Our routine uses its full Krawtchouk and Hahn tables.

At $\ell = 2$, Chang et al.’s specialized method is fastest when preparation is included, while at $\ell = 4$, first use runtimes are closer. At $\ell = 6, 8$, our conversion has the lowest first-use median throughout the tested range.

Common orbit-image preparation. For repeated conversions at the same n and weight ℓ , each method first computes the Schur images of all $\binom{\ell+2}{2}$ weight- ℓ orbit sums. Subsequent conversions then use the same routine to combine these images with the new coefficients. At fixed ℓ , this gives $O(n^2)$ arithmetic per conversion.

Preparing all orbit images may cost more than converting one operator directly. [Figure C.2](#) therefore reports one first-use timing as well as cached conversion, using the same inputs as [Figure C.1](#). Optimized Anschuetz has the lowest median for preparation and one conversion, except at $\ell = 2$, $n = 40$, where Chang’s construction is faster. The cached conversion times are comparable, as is expected.

In both comparisons, all three methods agree with an independent reference computation to relative Hilbert–Schmidt error below 5×10^{-15} . Further implementation details and benchmark data are available in the GitHub repository [[Sau26](#)].

C.2.5 Accuracy of the coordinate conversions

We check all six conversion directions against independent references, where we use the full matrices for $n = 0, \dots, 6$ and partial inputs all the way up to $n = 40$. Relative Hilbert–Schmidt errors remain below 2.5×10^{-15} in the small-system checks and the larger Hahn tests. Some shared-factor tests at $n = 30$ and 40 exceed the chosen tolerances, with relative errors up to 5.2×10^{-8} , which is still fairly small. Details of the test inputs, references and results are available in the GitHub repository [[Sau26](#)]. Application-specific checks are reported in [Section C.3.1](#).

C.3 Application benchmark details

The random-input benchmarks in [Section 6.3](#) compare our conversion to optimized Anschuetz with the same inputs, which for invariant dynamics are random Hamiltonians, initial states and observables.

Calculations run on one numerical-library thread on an AMD Ryzen 7 7735U. For each tested size between $n = 2$ and 20 we use three independent inputs and five timed repetitions per input after one warmup. We report medians and interquartile ranges of these results. For dynamics, the reported times also include the evaluation of [Equation \(6.13\)](#) at $L = 241$ equally spaced times in $[0, 12]$. Input generation and validation are excluded from measurement. A separate sweep varies L at $n = 12$, to check the benefit of our algorithm for a growing number of evaluation times.

The spectral comparison includes one diagonalization algorithm (not adapted to permutation symmetry) at selected sizes through $n = 10$.

For the Ising model, [Figure 6.4](#) compares our conversion with a direct construction using PIQS. We measure seven repetitions after one warmup at selected sizes through $n = 40$, reporting preparation and total runtime separately.

Implementation details are given in [Section C.2.1](#) and the GitHub repository [\[Sau26\]](#).

C.3.1 Application accuracy

Independent checks test the Hahn and shared-factor implementations against full-matrix references for $n = 2, \dots, 6$ and collective-spin Ising references at selected sizes through $n = 40$. All 48 configurations pass the chosen tolerances. The maximum absolute errors are 1.8×10^{-15} for the small-system spectra and 1.9×10^{-13} for expectation curves on the stated time grid.

The additional spectral and Ising comparisons also pass the 10^{-8} threshold for relative spectrum errors and absolute magnetization errors in all 405 and 140 trials, respectively. The largest relative spectrum error is 8.9×10^{-15} , and the largest absolute magnetization error is 3.8×10^{-15} . These results concern the tested inputs and times. Further details of the separate accuracy checks are available in the GitHub repository [\[Sau26\]](#).

C.4 Additional benchmark figures

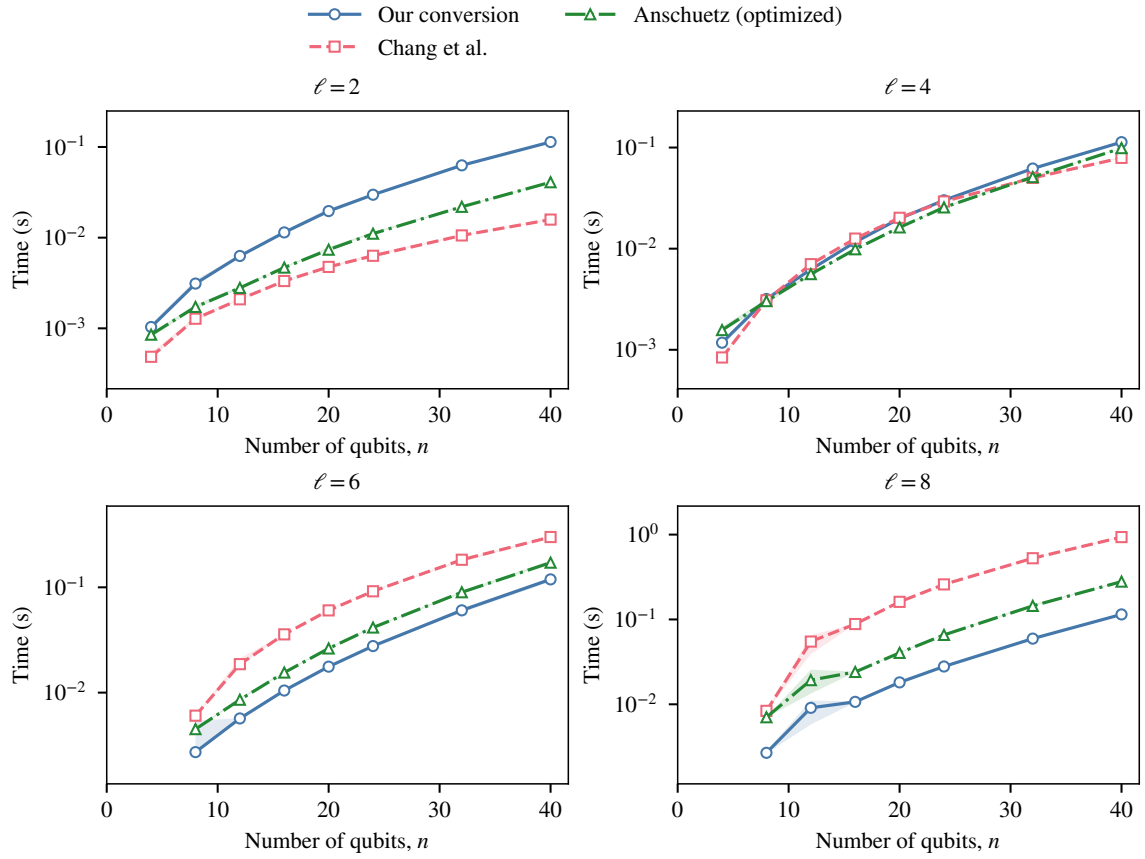


Figure C.1: Fixed-locality Schur construction. All three methods return the nonzero entries of the distinct Schur blocks. Each timing includes all preparation and one conversion, as described in [Section C.2.4](#). Markers and shading show medians and interquartile ranges.

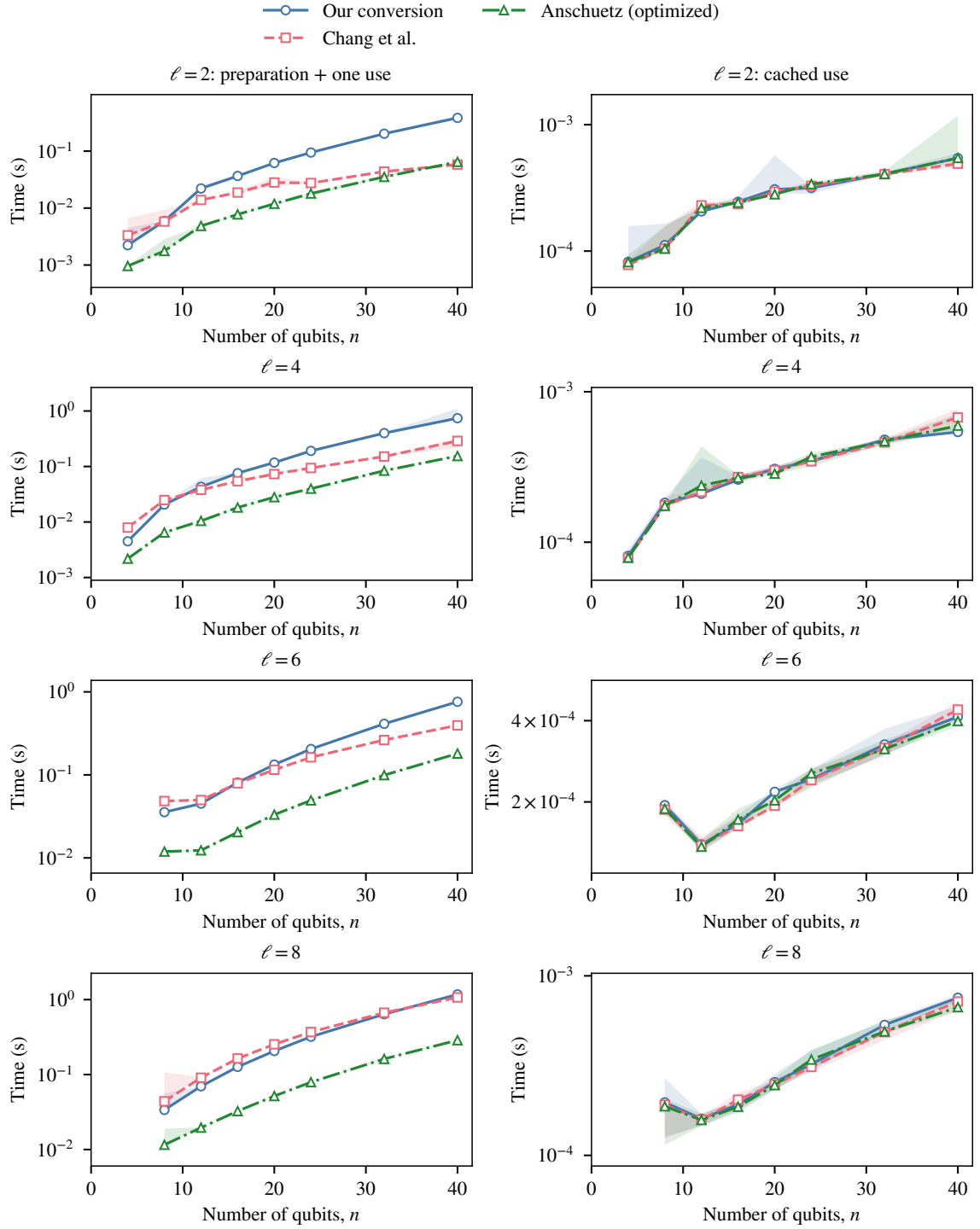


Figure C.2: Common orbit-image preparation at weights $\ell = 2, 4, 6, 8$ (top to bottom). Left: Including preparation. Right: Excluding preparation in accordance with [Section C.2.4](#). Markers and shading show medians and interquartile ranges.

AI Disclosure

We used LLM-based tools to assist with the creation of this document. The usage of these tools was limited to the following:

- Literature review and research
- Grammatical and small stylistic corrections and suggestions as well as help with LaTeX formatting
- The main mathematical contributions were made by the author; however, LLMs were helpful in finding the connections to prior results and their potential applications, especially in regard to the Krawtchouk and Hahn polynomials and transforms
- Proofreading and improving upon smaller mathematical arguments
- Writing parts of the Python code for implementation and optimization as well as for the generation of figures and the management of the GitHub repository

We verified the correctness of all mathematical results and proofs, and the code was tested for correctness and performance. The author takes full responsibility for the content of this document. Access to the concrete models used and chat logs is available upon request.

Acknowledgments

I want to thank Professor Dr. Mario Berta for the opportunity to work on this interesting and resourceful topic. For their continued support and encouragement, I want to thank my advisors Dr. Robert Salzmänn and Julius Zeiss. Without their helpful steering, this thesis would not have been possible. I also want to thank Professor Philipp Feinsilver who provided me with helpful material regarding the Krawtchouk polynomials. I want to thank my family and friends for their support and patience during my studies and the writing of this thesis.

Special thanks go to Sinan Blanck and Friedrich Kurth for their helpful discussions regarding modern Panov theory. Philipp Oldigs was crucial in the investigation of extraterrestrial horses. Yusuf Yilmaz provided transportation, culinary and (artificial) intelligence support during the writing of this thesis. Finally, Bastian Seibel provided helpful insights and discussions regarding Tartakower theory.